8.2 Query, key, value, and attention

Attention is a strategy for processing global information efficiently, focusing just on the parts of the signal that are most salient to the task at hand.

What we present below is the so-called "dot-product attention" mechanism; there can be other variants that involve more complex attention functions

It might help our understanding of the "attention" mechanism to think about a dictionary look-up scenario. Consider a dictionary with keys k mapping to some values $v(k)$. For example, let k be the name of some foods, such as `pizza`, `apple`, `sandwich`, `donut`, `chili`, `burrito`, `sushi`, `hamburger`, ... The corresponding values may be information about the food, such as where it is available, how much it costs, or what its ingredients are.

Suppose that instead of looking up foods by a specific name, we wanted to query by cuisine, e.g., "`mexican`" foods. Clearly, we cannot simply look for the word "`mexican`" among the dictionary keys, since that word is not a food. What does work is to utilize again the idea of finding "similarity" between vector embeddings of the query and the keys. The end result we'd hope to get, is a probability distribution over the foods, $p(k|q)$ indicating which are best matches for a given query q. With such a distribution, we can look for keys that are semantically close to the given query.

More concretely, to get such distribution, we follow these steps: First, embed the word we are interested in ("`mexican`" in our example) into a so-called query vector, denoted simply as $q \in \mathbb{R}^{d_k \times 1}$ where d_k is the embedding dimension.

Next, suppose our given dictionary has n number of entries/entries, we embed each one of these into a so-called key vector. In particular, for each of the j^{th} entry in the dictionary, we produce a $k_j \in \mathbb{R}^{d_k \times 1}$ key vector, where $j = 1, 2, 3, \ldots, n$.

We can then obtain the desired probability distribution using a softmax (see Chapter 6) applied to the inner-product between the key and query:

$$p(k|q) = softmax\left([q^T k_1; q^T k_2; q^T k_3; \ldots, q^T k_n]\right)$$

This vector-based lookup mechanism has come to be known as "attention" in the sense that $p(k|q)$ is a conditional probability distribution that says how much attention should be given to the key k_j for a given query q.

In other words, the conditional probability distribution $p(k|q)$ gives the "attention weights," and the weighted average value

$$\sum_j p(k_j|q)\, v_j \tag{8.3}$$

is the "attention output."

The meaning of this weighted average value may be ambiguous when the values are just words. However, the attention output really becomes meaningful when the value are projected in some semantic embedding space (and such projection are typically done in transformers via learned embedding weights).

The same weighted-sum idea generalizes to multiple query, key, and values. In particular, suppose there are n_q number of queries, n_k number of keys (and therefore n_k number of values), one can compute an attention matrix

$$A = \begin{bmatrix} \text{softmax}\left(\left[q_1^\top k_1 & q_1^\top k_2 & \cdots & q_1^\top k_{n_k}\right]/\sqrt{d_k}\right) \\ \text{softmax}\left(\left[q_2^\top k_1 & q_2^\top k_2 & \cdots & q_2^\top k_{n_k}\right]/\sqrt{d_k}\right) \\ \vdots \\ \text{softmax}\left(\left[q_{n_q}^\top k_1 & q_{n_q}^\top k_2 & \cdots & q_{n_q}^\top k_{n_k}\right]/\sqrt{d_k}\right) \end{bmatrix} \tag{8.4}$$

Here, softmax_j is a softmax over the n_k-dimensional vector indexed by j, so in Eq. 8.2 this means a softmax computed over keys. In this equation, the normalization by $\sqrt{d_k}$ is

done to reduce the magnitude of the dot product, which would otherwise grow undesirably large with increasing d_k, making it difficult for (overall) training.

Let α_{ij} be the entry in ith row and jth column in the attention matrix A. Then α_{ij} helps answer the question "which tokens $x^{(j)}$ help the most with predicting the corresponding output token $y^{(i)}$?" The attention output is given by a weighted sum over the values:

$$y^{(i)} = \sum_{j=1}^{n} \alpha_{ij} v_j$$

8.2.1 Self Attention

Self-attention is an attention mechanism where the keys, values, and queries are all generated from the same input.

At a very high level, typical transformer with self-attention layers maps $\mathbb{R}^{n \times d} \longrightarrow \mathbb{R}^{n \times d}$. In particular, the transformer takes in data (a sequence of tokens) $X \in \mathbb{R}^{n \times d}$ and for each token $x^{(i)} \in \mathbb{R}^{d \times 1}$, it computes (via learned projection, to be discussed in Section 8.3.1), a query $q_i \in \mathbb{R}^{d_q \times 1}$, key $k_i \in \mathbb{R}^{d_k \times 1}$, and value $v_i \in \mathbb{R}^{d_v \times 1}$. In practice, $d_q = d_k = d_v$ and we often denote all three embedding dimension via a unified d_k.

Note that d_k differs from d: d is the dimension of raw input token $\in \mathbb{R}^{d_q \times 1}$

The self-attention layer then take in these query, key, and values, and compute a self-attention matrix

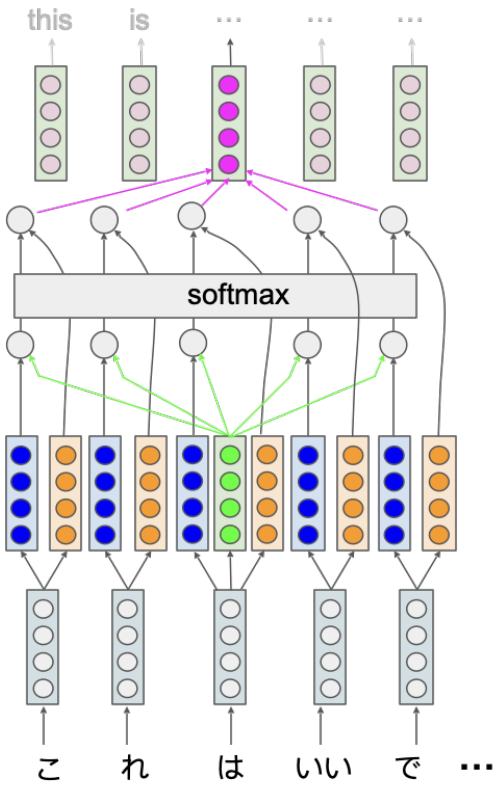
$$A = \begin{bmatrix} \text{softmax}\left(\begin{bmatrix} q_1^\top k_1 & q_1^\top k_2 & \cdots & q_1^\top k_n \end{bmatrix} / \sqrt{d_k}\right) \\ \text{softmax}\left(\begin{bmatrix} q_2^\top k_1 & q_2^\top k_2 & \cdots & q_2^\top k_n \end{bmatrix} / \sqrt{d_k}\right) \\ \vdots \\ \text{softmax}\left(\begin{bmatrix} q_n^\top k_1 & q_n^\top k_2 & \cdots & q_n^\top k_n \end{bmatrix} / \sqrt{d_k}\right) \end{bmatrix} \tag{8.5}$$

Comparing this self-attention matrix with the attention matrix described in Equation 8.2, we notice the only difference lies in the dimensions: since in self-attention, the query, key, and value all come from the same input, we have $n_q = n_k = n_v$, and we often denote all three with a unified n.

The self-attention output is then given by a weighted sum over the values:

$$y^{(i)} = \sum_{j=1}^{n} \alpha_{ij} v_j$$

This diagram below shows (only) the middle input token generating a query that is then combined with the keys computed with all tokens to generate the attention weights via a softmax. The output of the softmax is then combined with values computed from all tokens, to generate the attention output corresponding to the middle input token. Repeating this for each input token then generates the output.



Study Question: We have five colored tokens in the diagram above (gray, blue, organge, green, red). Could you read off the diagram the correspondence between the color and input, query, query, value, output?

Note that the size of the output is the same as the size of the input. Also, observe that there is no apparent notion of ordering of the input words in the depicted structure. Positional information can be added by encoding a number for token (giving say, the token's position relative to the start of the sequence) into the vector embedding of each token. And note that a given query need not pay attention to all other tokens in the input; in this example, the token used for the query is not used for a key or value.

More generally, a *mask* may be applied to limit which tokens are used in the attention computation. For example, one common mask limits the attention computation to tokens that occur previously in time to the one being used for the query. This prevents the attention mechanism from "looking ahead" in scenarios where the transformer is being used to generate one token at a time.

Each self-attention stage is trained to have key, value, and query embeddings that lead it to pay specific attention to some particular feature of the input. We generally want to pay attention to many different kinds of features in the input; for example, in translation one feature might be be the verbs, and another might be objects or subjects. A transformer utilizes multiple instances of self-attention, each known as an "attention head," to allow combinations of attention paid to many different features.

8.3 Transformers

A transformer is the composition of a number of transformer blocks, each of which has multiple attention heads. At a very high-level, the goal of a transformer block is to output

a really rich, useful representation for each input token, all for the sake of being high-performant for whatever task the model is trained to learn.

Rather than depicting the transformer graphically, it is worth returning to the beauty of the underlying equations[1].

8.3.1 Learned embedding

For simplicity, we assume the transformer internally uses self-attention. Full general attention layers work out similarly.

Formally, a transformer block is a parameterized function f_θ that maps $\mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$, where the input data $X \in \mathbb{R}^{n\times d}$ is often represented as a sequence of n tokens, with each token $x^{(i)} \in \mathbb{R}^{d\times 1}$.

Three projection matrices (weights) W_q, W_k, W_v are to be learned, such that, for each token $x^{(i)} \in \mathbb{R}^{d\times 1}$, we produce 3 distinct vectors: a query vector $q_i = W_q^T x^{(i)}$; a key vector $k_i = W_k^T x^{(i)}$; a value vector $v_i = W_v^T x^{(i)}$, all 3 of these vectors $\mathbb{R}^{d_k\times 1}$ and the learned weights $W_q, W_k, W_v \in \mathbb{R}^{d\times d_k}$.

If we stack these n query, key, value vectors into matrix- form, such that $Q \in \mathbb{R}^{n\times d_k}$, $K \in \mathbb{R}^{n\times d_k}$, and $V \in \mathbb{R}^{n\times d_k}$, then we can more compactly write out the learned transformation from the sequence of input token X:

$$\begin{aligned} Q &= XW_q \\ K &= XW_k \\ V &= XW_v \end{aligned}$$

These Q, K, V triple can then be used to produce one (self)attention-layer output. One such layer is called one "attention head".

One can have more than one "attention head", such that: the queries, keys, and values are embedded via encoding matrices:

$$Q^{(h)} = XW_{h,q} \tag{8.6}$$
$$K^{(h)} = XW_{h,k} \tag{8.7}$$
$$V^{(h)} = XW_{h,v} \tag{8.8}$$

and $W_{h,q}, W_{h,k}, W_{h,v} \in \mathbb{R}^{d\times d_k}$ where d_k is the size of the key/query embedding space, and $h \in \{1, \cdots, H\}$ is an index over "attention heads."

for each attention-head h, we learn one set of $W_{h,q}, W_{h,k}, W_{h,v}$.

We then perform a weighted sum over all the outputs for each head,

$$u'^{(i)} = \sum_{h=1}^{H} W_{h,c}^T \sum_{j=1}^{n} \alpha_{ij}^{(h)} V_j^{(h)}, \tag{8.9}$$

where $W_{h,c} \in \mathbb{R}^{d_k\times d}$, $u'^{(i)} \in \mathbb{R}^{d\times 1}$, the indices $i \in \{1, \cdots, n\}$ and $j \in \{1, \cdots, n\}$ are an integer index over tokens.

$V_j^{(h)}$ is the $d_k \times 1$ value embedding vector that corresponds to the input token x^j for attention head h.

This is then standardized and combined with $x^{(i)}$ using a LayerNorm function (defined below) to become

$$u^{(i)} = \text{LayerNorm}\left(x^{(i)} + u'^{(i)}; \gamma_1, \beta_1\right) \tag{8.10}$$

with parameters $\gamma_1, \beta_1 \in \mathbb{R}^d$.

[1] The presentation here follows the notes by John Thickstun.

To get the final output, we follow the "intermediate output then layer norm" recipe again. In particular, we first get the transformer block output $z'^{(i)}$ given by

$$z'^{(i)} = W_2^T \text{ReLU}\left(W_1^T u^{(i)}\right) \tag{8.11}$$

with weights $W_1 \in \mathbb{R}^{d\times m}$ and $W_2 \in \mathbb{R}^{m\times d}$. This is then standardized and combined with $u^{(i)}$ to give the final output $z^{(i)}$:

$$z^{(i)} = \text{LayerNorm}\left(u^{(i)} + z'^{(i)}; \gamma_2, \beta_2\right), \tag{8.12}$$

with parameters $\gamma_2, \beta_2 \in \mathbb{R}^d$. These vectors are then assembled (e.g., through parallel computation) to produce $z \in \mathbb{R}^{n\times d}$.

The LayerNorm function transforms a d-dimensional input z with parameters $\gamma, \beta \in \mathbb{R}^d$ into

$$\text{LayerNorm}(z; \gamma, \beta) = \gamma\frac{z - \mu_z}{\sigma_z} + \beta, \tag{8.13}$$

where μ_z is the mean and σ_z the standard deviation of z:

$$\mu_z = \frac{1}{d}\sum_{i=1}^{d} z_i \tag{8.14}$$

$$\sigma_z = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(z_i - \mu_z)^2}. \tag{8.15}$$

Layer normalization is done to improve convergence stability during training.

The model parameters comprise the weight matrices $W_{h,q}, W_{h,k}, W_{h,v}, W_{h,c}, W_1, W_2$ and the LayerNorm parameters $\gamma_1, \gamma_2, \beta_1, \beta_2$. A *transformer* is the composition of L transformer blocks, each with its own parameters:

$$f_{\theta_L} \circ \cdots \circ f_{\theta_2} \circ f_{\theta_1}(x) \in \mathbb{R}^{n\times d}. \tag{8.16}$$

The hyperparameters of this model are d, d_k, m, H, and L.

8.3.2 Variations and training

Many variants on this transformer structure exist. For example, the LayerNorm may be moved to other stages of the neural network. Or a more sophisticated attention function may be employed instead of the simple dot product used in Eq. 8.2. Transformers may also be used in pairs, for example, one to process the input and a separate one to generate the output given the transformed input. Self-attention may also be replaced with cross-attention, where some input data are used to generate queries and other input data generate keys and values. Positional encoding and masking are also common, though they are left implicit in the above equations for simplicity.

How are transformers trained? The number of parameters in θ can be very large; modern transformer models like GPT4 have tens of billions of parameters or more. A great deal of data is thus necessary to train such models, else the models may simply overfit small datasets.

Training large transformer models is thus generally done in two stages. A first "pre-training" stage employs a very large dataset to train the model to extract patterns. This is done with unsupervised (or self-supervised) learning and unlabelled data. For example, the well-known BERT model was pre-trained using sentences with words masked. The

model was trained to predict the masked words. BERT was also trained on sequences of sentences, where the model was trained to predict whether two sentences are likely to be contextually close together or not. The pre-training stage is generally very expensive.

The second "fine-tuning" stage trains the model for a specific task, such as classification or question answering. This training stage can be relatively inexpensive, but it generally requires labeled data.

CHAPTER 9

Non-parametric methods

Neural networks have adaptable complexity, in the sense that we can try different structural models and use cross validation to find one that works well on our data. Beyond neural networks, we may further broaden the class of models that we can fit to our data, for example as illustrated by the techniques introduced in this chapter.

Here, we turn to models that automatically adapt their complexity to the training data. The name *non-parametric methods* is misleading: it is really a class of methods that does not have a fixed parameterization in advance. Rather, the complexity of the parameterization can grow as we acquire more data.

Some non-parametric models, such as nearest-neighbor, rely directly on the data to make predictions and do not compute a model that summarizes the data. Other non-parametric methods, such as decision trees, can be seen as dynamically constructing something that ends up looking like a more traditional parametric model, but where the actual training data affects exactly what the form of the model will be.

These are sometimes called *classification trees*; the decision analysis literature uses "decision tree" for a structure that lays out possible future events that consist of choices interspersed with chance nodes.

The non-parametric methods we consider here tend to have the form of a composition of simple models:

- *Nearest neighbor models*: (Section 9.1) where we don't process data at training time, but do all the work when making predictions, by looking for the closest training example(s) to a given new data point.
- *Tree models*: (Section 9.2) where we partition the input space and use different simple predictions on different regions of the space; the hypothesis space can become arbitrarily large allowing finer and finer partitions of the input space.
- *Ensemble models*: (Section 9.2.3) in which we train several different classifiers on the whole space and average the answers; this decreases the estimation error. In particular, we will look at bootstrap aggregation, or *bagging* of trees.
- *Boosting* is a way to construct a model composed of a sequence of component models (e.g., a model consisting of a sequence of trees, each subsequent tree seeking to correct errors in the previous trees) that decreases both estimation and structural error. We won't consider this in detail in this class.

Why are we studying these methods, in the heyday of complicated models such as neural networks?

- They are fast to implement and have few or no hyperparameters to tune.
- They often work as well as or better than more complicated methods.
- Predictions from some of these models can be easier to explain to a human user: decision trees are fairly directly human-interpretable, and nearest neighbor methods can justify their decision to some extent by showing a few training examples that the prediction was based on.

9.1 Nearest Neighbor

In nearest-neighbor models, we don't do any processing of the data at training time – we just remember it! All the work is done at prediction time.

Input values x can be from any domain $\mathcal{X}$ ($\mathbb{R}^d$, documents, tree-structured objects, etc.). We just need a distance metric, $d : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^+$, which satisfies the following, for all $x, x', x'' \in \mathcal{X}$:

$$\begin{aligned} d(x, x) &= 0 \\ d(x, x') &= d(x', x) \\ d(x, x'') &\leqslant d(x, x') + d(x', x'') \end{aligned}$$

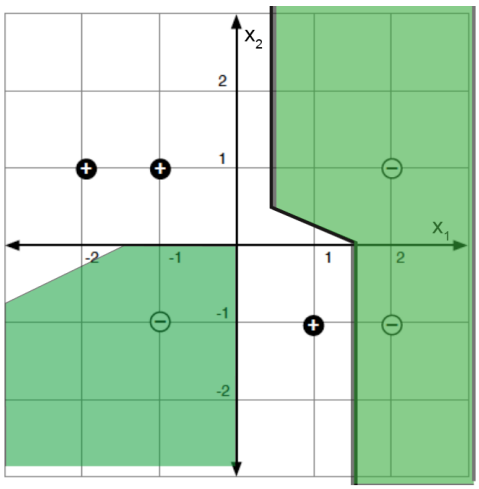
Given a data-set $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$, our predictor for a new $x \in \mathcal{X}$ is

$$h(x) = y^{(i)} \quad \text{where} \quad i = \arg\min_i d(x, x^{(i)}) \ , \tag{9.1}$$

that is, the predicted output associated with the training point that is closest to the query point x. Tie breaking is typically done at random.

This same algorithm works for regression *and* classification!

The nearest neighbor prediction function can be described by dividing the space up into regions whose closest point is each individual training point as shown below :



Decision boundary regions can also be described by Voronoi diagrams. In a Voronoi diagram, each of the data points would have its own "cell" or region in the space that is closest to the data point in question. In the diagram provided here, cells have been merged if the predicted value is the same in adjacent cells.

In each region, we predict the associated y value.

Study Question: Convince yourself that these boundaries do represent the nearest-neighbor classifier derived from these six data points.

There are several useful variations on this method. In k-*nearest-neighbors*, we find the k training points nearest to the query point x and output the majority y value for classification or the average for regression. We can also do *locally weighted regression* in which we

fit locally linear regression models to the k nearest points, possibly giving less weight to those that are farther away. In large data-sets, it is important to use good data structures (e.g., ball trees) to perform the nearest-neighbor look-ups efficiently (without looking at all the data points each time).

9.2 Tree Models

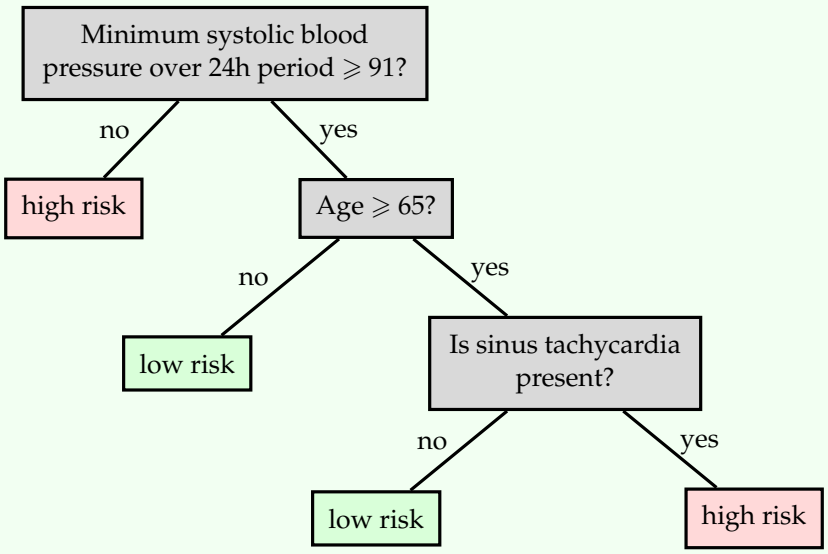
The idea here is that we would like to find a partition of the input space and then fit very simple models to predict the output in each piece. The partition is described using a (typically binary) "tree" that recursively splits the space.

Tree methods differ by:

- The class of possible ways to split the space at each node; these are typically linear splits, either aligned with the axes of the space, or sometimes using more general classifiers.
- The class of predictors within the partitions; these are often simply constants, but may be more general classification or regression models.
- The way in which we control the complexity of the hypothesis: it would be within the capacity of these methods to have a separate partition element for each individual training example.
- The algorithm for making the partitions and fitting the models.

One advantage of tree models is that they are easily interpretable by humans. This is important in application domains, such as medicine, where there are human experts who often ultimately make critical decisions and who need to feel confident in their understanding of recommendations made by an algorithm. Below is an example decision tree, illustrating how one might be able to understand the decisions made by the tree.

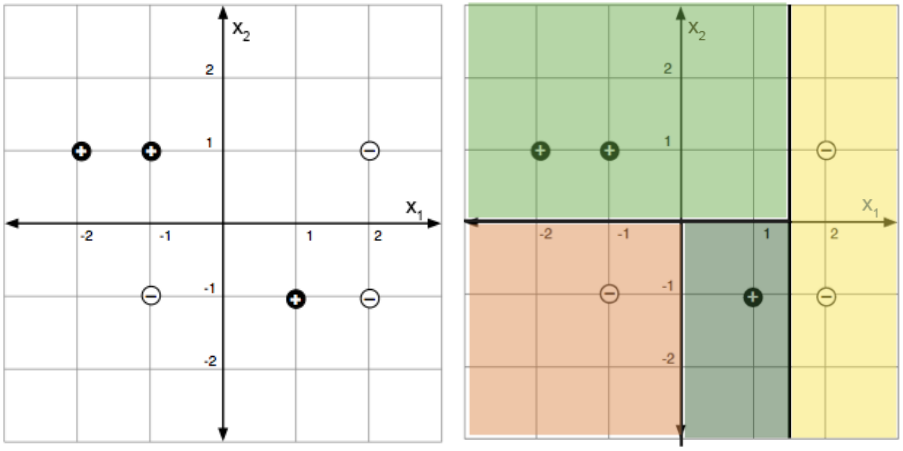
Example: Here is a sample tree (reproduced from Breiman, Friedman, Olshen, Stone (1984)):



These methods are most appropriate for domains where the input space is not very high-dimensional and where the individual input features have some substantially useful information individually or in small groups. Trees would not be good for image input, but might be good in cases with, for example, a set of meaningful measurements of the condition of a patient in the hospital, as in the example above.

We'll concentrate on the CART/ID3 ("classification and regression trees" and "iterative dichotomizer 3", respectively) family of algorithms, which were invented independently in the statistics and the artificial intelligence communities. They work by greedily constructing a partition, where the splits are *axis aligned* and by fitting a *constant* model in the leaves. The interesting questions are how to select the splits and how to control complexity. The regression and classification versions are very similar.

As a concrete example, consider the following images:



The left image depicts a set of labeled data points in a two-dimensional feature space. The right shows a partition into regions by a decision tree, in this case having no classification errors in the final partitions.

9.2.1 Regression

The predictor is made up of

- a partition function, π, mapping elements of the input space into exactly one of M regions, $R_1, \ldots, R_M$, and
- a collection of M output values, O_m, one for each region.

If we already knew a division of the space into regions, we would set O_m, the constant output for region R_m, to be the average of the training output values in that region. For a training data set $\mathcal{D} = \left\{\left(x^{(i)}, y^{(i)}\right)\right\}, i = 1, \ldots n$, we let I be an indicator set of all of the elements within $\mathcal{D}$, so that $I = \{1, \ldots, n\}$ for our whole data set. We can define I_m as the subset of data set samples that are in region R_m, so that $I_m = \{i \mid x^{(i)} \in R_m\}$. Then

$$O_m = \text{average}_{i \in I_m} \; y^{(i)} \quad .$$

We can define the error in a region as E_m. For example, E_m as the sum of squared error would be expressed as

$$E_m = \sum_{i \in I_m} (y^{(i)} - O_m)^2 \quad . \tag{9.2}$$

Ideally, we would select the partition to minimize

$$\lambda M + \sum_{m=1}^{M} E_m \quad , \tag{9.3}$$

for some regularization constant λ. It is enough to search over all partitions of the training data (not all partitions of the input space!) to optimize this, but the problem is NP-complete.

Study Question: Be sure you understand why it's enough to consider all partitions of the training data, if this is your objective.

9.2.1.1 Building a tree

So, we'll be greedy. We establish a criterion, given a set of data, for finding the best single split of that data, and then apply it recursively to partition the space. For the discussion below, we will select the partition of the data that *minimizes the sum of the sum of squared errors of each partition element.* Then later, we will consider other splitting criteria.

Given a data set $\mathcal{D} = \left\{\left(x^{(i)}, y^{(i)}\right)\right\}, i = 1, \ldots n$, we now consider I to be an indicator of the subset of elements within $\mathcal{D}$ that we wish to build a tree (or subtree) for. That is, I may already indicate a subset of data set $\mathcal{D}$, based on prior splits in constructing our overall tree. We define terms as follows:

- $I^+_{j,s}$ indicates the set of examples (subset of I) whose feature value in dimension j is greater than or equal to split point s;
- $I^-_{j,s}$ indicates the set of examples (subset of I) whose feature value in dimension j is less than s;
- $\hat{y}^+_{j,s}$ is the average y value of the data points indicated by set $I^+_{j,s}$; and
- $\hat{y}^-_{j,s}$ is the average y value of the data points indicated by set $I^-_{j,s}$.

Here is the pseudocode. In what follows, k is the largest leaf size that we will allow in the tree, and is a hyperparameter of the algorithm.

BUILDTREE(I, k)

if $|I| \leqslant k$
  Set $\hat{y} = \text{average}_{i \in I}\, y^{(i)}$
  return LEAF(value $= \hat{y}$)
else
  for each split dimension j and split value s
    Set $I^+_{j,s} = \{i \in I \mid x^{(i)}_j \geqslant s\}$
    Set $I^-_{j,s} = \{i \in I \mid x^{(i)}_j < s\}$
    Set $\hat{y}^+_{j,s} = \text{average}_{i \in I^+_{j,s}}\, y^{(i)}$
    Set $\hat{y}^-_{j,s} = \text{average}_{i \in I^-_{j,s}}\, y^{(i)}$
    Set $E_{j,s} = \sum_{i \in I^+_{j,s}} (y^{(i)} - \hat{y}^+_{j,s})^2 + \sum_{i \in I^-_{j,s}} (y^{(i)} - \hat{y}^-_{j,s})^2$
  Set $(j^*, s^*) = \arg\min_{j,s} E_{j,s}$
return NODE$(j^*, s^*,$ BUILDTREE$(I^-_{j^*,s^*}, k),$ BUILDTREE$(I^+_{j^*,s^*}, k))$

In practice, we typically start by calling BUILDTREE with the first input equal to our whole data set (that is, with $I = \{1, \ldots, n\}$). But then that call of BUILDTREE can recursively lead to many other calls of BUILDTREE.

Let's think about how long each call of BUILDTREE takes to run. We have to consider all possible splits. So we consider a split in each of the d dimensions. In each dimension, we only need to consider splits betwee two data points (any other split will give the same error on the training data). So, in total, we consider $O(dn)$ splits in each call to BUILDTREE.

Study Question: Concretely, what would be a good set of split-points to consider for dimension j of a data set indicated by I?

9.2.1.2 Pruning

It might be tempting to regularize by using a somewhat large value of k, or by stopping when splitting a node does not significantly decrease the error. One problem with short-sighted stopping criteria is that they might not see the value of a split that will require one more split before it seems useful.

Study Question: Apply the decision-tree algorithm to the XOR problem in two dimensions. What is the training-set error of all possible hypotheses based on a single split?

So, we will tend to build a tree that is too large, and then prune it back.

We define *cost complexity* of a tree T, where m ranges over its leaves, as

$$C_\alpha(T) = \sum_{m=1}^{|T|} E_m(T) + \alpha|T| \ , \tag{9.4}$$

and $|T|$ is the number of leaves. For a fixed α, we can find a T that (approximately) minimizes $C_\alpha(T)$ by "weakest-link" pruning:

- Create a sequence of trees by successively removing the bottom-level split that minimizes the increase in overall error, until the root is reached.
- Return the T in the sequence that minimizes the cost complexity.

We can choose an appropriate α using cross validation.

9.2.2 Classification

The strategy for building and pruning classification trees is very similar to the strategy for regression trees.

Given a region R_m corresponding to a leaf of the tree, we would pick the output class y to be the value that exists most frequently (the *majority value*) in the data points whose x values are in that region, i.e., data points indicated by I_m:

$$O_m = \text{majority}_{i \in I_m} \ y^{(i)} \ .$$

Let's now define the error in a region as the number of data points that do not have the value O_m:

$$E_m = \left| \{ i \mid i \in I_m \text{ and } y^{(i)} \neq O_m \} \right| \ .$$

We define the *empirical probability* of an item from class k occurring in region m as:

$$\hat{P}_{m,k} = \hat{P}(I_m, k) = \frac{\left| \{ i \mid i \in I_m \text{ and } y^{(i)} = k \} \right|}{N_m} \ ,$$

where N_m is the number of training points in region m; that is, $N_m = |I_m|$. For later use, we'll also define the empirical probabilities of split values, $\hat{P}_{m,j,s}$, as the fraction of points with dimension j in split s occurring in region m (one branch of the tree), and $1 - \hat{P}_{m,j,s}$ as the complement (the fraction of points in the other branch).

Splitting criteria In our greedy algorithm, we need a way to decide which split to make next. There are many criteria that express some measure of the "impurity" in child nodes. Some measures include:

- *Misclassification error*:
$$Q_m(T) = \frac{E_m}{N_m} = 1 - \hat{P}_{m,O_m} \tag{9.5}$$
- *Gini index*:
$$Q_m(T) = \sum_k \hat{P}_{m,k}(1 - \hat{P}_{m,k}) \tag{9.6}$$
- *Entropy*:
$$Q_m(T) = H(I_m) = -\sum_k \hat{P}_{m,k} \log_2 \hat{P}_{m,k} \tag{9.7}$$

  So that the entropy H is well-defined when $\hat{P} = 0$, we will stipulate that $0 \log_2 0 = 0$.

These splitting criteria are very similar, and it's not entirely obvious which one is better. We will focus on entropy, just to be concrete.

Analogous to how for regression we choose the dimension j and split s that minimizes the sum of squared error $E_{j,s}$, for classification, we choose the dimension j and split s that minimizes the weighted average entropy over the "child" data points in each of the two corresponding splits, $I^+_{j,s}$ and $I^-_{j,s}$. We calculate the entropy in each split based on the empirical probabilities of class memberships in the split, and then calculate the weighted average entropy $\hat{H}$ as

$$\begin{aligned}\hat{H} &= (\text{fraction of points in left data set}) \cdot H(I^-_{j,s}) \\ &\quad + (\text{fraction of points in right data set}) \cdot H(I^+_{j,s}) \\ &= (1 - \hat{P}_{m,j,s}) H(I^-_{j,s}) + \hat{P}_{m,j,s} H(I^+_{j,s}) \\ &= \frac{|I^-_{j,s}|}{N_m} \cdot H(I^-_{j,s}) + \frac{|I^+_{j,s}|}{N_m} \cdot H(I^+_{j,s}) \,. \end{aligned} \tag{9.8}$$

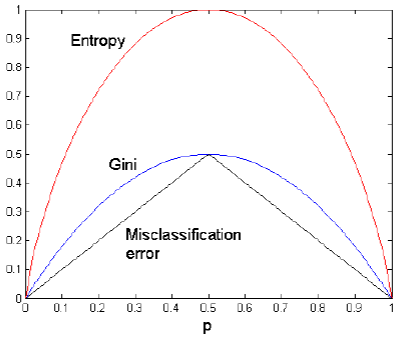
Choosing the split that minimizes the entropy of the children is equivalent to maximizing the *information gain* of the test $x_j = s$, defined by

$$\text{INFOGAIN}(x_j = s, I_m) \quad = \quad H(I_m) - \left(\frac{|I^-_{j,s}|}{N_m} \cdot H(I^-_{j,s}) + \frac{|I^+_{j,s}|}{N_m} \cdot H(I^+_{j,s}) \right) \tag{9.9}$$

In the two-class case (with labels 0 and 1), all of the splitting criteria mentioned above have the values

$$\begin{cases} 0.0 & \text{when } \hat{P}_{m,0} = 0.0 \\ 0.0 & \text{when } \hat{P}_{m,0} = 1.0 \end{cases} .$$

The respective impurity curves are shown below, where $p = \hat{P}_{m,0}$; the vertical axis plots $Q_m(T)$ for each of the three criteria.



There used to be endless haggling about which impurity function one should use. It seems to be traditional to use *entropy* to select which node to split while growing the tree, and *misclassification error* in the pruning criterion.

9.2.3 Bagging

One important limitation or drawback in conventional trees is that they can have high estimation error: small changes in the data can result in very big changes in the resulting tree.

Bootstrap aggregation is a technique for reducing the estimation error of a non-linear predictor, or one that is adaptive to the data. The key idea applied to trees, is to build multiple trees with different subsets of the data, and then create an ensemble model that combines the results from multiple trees to make a prediction.

- Construct B new data sets of size n. Each data set is constructed by sampling n data points with replacement from $\mathcal{D}$. A single data set is called *bootstrap sample* of $\mathcal{D}$.
- Train a predictor $\hat{f}^b(x)$ on each bootstrap sample.
- *Regression case*: bagged predictor is

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^b(x) \ . \tag{9.10}$$

- *Classification case*: Let K be the number of classes. We find a majority bagged predictor as follows. We let $\hat{f}^b(x)$ be a "one-hot" vector with a single 1 and $K-1$ zeros, and define the predicted output $\hat{y}$ for predictor f^b as $\hat{y}^b(x) = \arg\max_k \hat{f}^b(x)_k$. Then

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^b(x), \tag{9.11}$$

  which is a vector containing the proportion of classifiers that predicted each class k for input x. Then the overall predicted output is

$$\hat{y}_{\text{bag}}(x) = \arg\max_k \hat{f}_{\text{bag}}(x)_k \ . \tag{9.12}$$

There are theoretical arguments showing that bagging does, in fact, reduce estimation error. However, when we bag a model, any simple intrepetability is lost.

9.2.4 Random Forests

Random forests are collections of trees that are constructed to be de-correlated, so that using them to vote gives maximal advantage. In competitions, they often have excellent classification performance among large collections of much fancier methods.

In what follows, B, m, and n are hyperparameters of the algorithm.

RANDOMFOREST($\mathcal{D}$; B, m, n)

```
for b = 1, ..., B
    Draw a bootstrap sample D_b of size n from D
    Grow a tree T_b on data D_b by recursively:
        Select m variables at random from the d variables
        Pick the best variable and split point among the m variables
        Split the node
return tree T_b
```

Given the ensemble of trees, vote to make a prediction on a new x.

9.2.5 Tree variants and tradeoffs

There are many variations on the tree theme. One is to employ different regression or classification methods in each leaf. For example, a linear regression might be used to model the examples in each leaf, rather than using a constant value.

In the relatively simple trees that we've considered, splits have been based on only a single feature at a time, and with the resulting splits being axis-parallel. Other methods for splitting are possible, including consideration of multiple features and linear classifiers based on those, potentially resulting in non-axis-parallel splits. Complexity is a concern in such cases, as many possible combinations of features may need to be considered, to select the best variable combination (rather than a single split variable).

Another generalization is a *hierarchical mixture of experts*, where we make a "soft" version of trees, in which the splits are probabilistic (so every point has some degree of membership in every leaf). Such trees can be trained using a form of gradient descent. Combinations of bagging, boosting, and mixture tree approaches (e.g., *gradient boosted trees*) and implementations are readily available (e.g., XGBoost).

Trees have a number of strengths, and remain a valuable tool in the machine learning toolkit. Some benefits include being relatively easy to interpret, fast to train, and ability to handle multi-class classification in a natural way. Trees can easily handle different loss functions; one just needs to change the predictor and loss being applied in the leaves. Methods also exist to identify which features are particularly important or influential in forming the tree, which can aid in human understanding of the data set. Finally, in many situations, trees perform surprisingly well, often comparable to more complicated regression or classification models. Indeed, in some settings it is considered good practice to start with trees (especially random forest or boosted trees) as a "baseline" machine learning model, against which one can evaluate performance of more sophisticated models.

CHAPTER 10

Markov Decision Processes

So far, most of the learning problems we have looked at have been *supervised*, that is, for each training input $x^{(i)}$, we are told which value $y^{(i)}$ should be the output. From a traditional machine-learning viewpoint, there're two other major groups of learning problems: one is the *unsupervised* learning problems, in which we are given data and no expected outputs, and we will look at later in Chapter 12.1 and Chapter 12.2.

The other major type is the so-called *Reinforcement learning* (RL) problems. Reinforcement learning differs significantly from supervised learning problems, and we will delve into the details later in in Chapter 11. However, it's worth pointing out one major difference at a very high level: in supervised learning, our goal is to learn a one-time static mapping to make predictions, whereas in RL, the setup requires us to sequentially take actions to maximize cumulative rewards.

This setup change necessitates additional mathematical and algorithmic tools for us to understand RL. *Markov decision process* (MDP) is precisely such a classical and fundamental tool.

10.1 Definition and value functions

Formally, a Markov decision process is $\langle \mathcal{S}, \mathcal{A}, T, R, \gamma \rangle$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and:

- $T : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is a *transition model*, where

$$T(s, a, s') = \Pr(S_t = s' | S_{t-1} = s, A_{t-1} = a) \ ,$$

  specifying a conditional probability distribution;

- $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is a *reward function*, where $R(s, a)$ specifies an immediate reward for taking action a when in state s; and

- $\gamma \in [0, 1]$ is a *discount factor*, which we'll discuss in Section 10.1.2.

The notation $S_t = s'$ uses a capital letter S to stand for a random variable, and small letter s to stand for a concrete value. So S_t here is a random variable that can take on elements of $\mathcal{S}$ as values.

In this class, we assume the rewards are deterministic functions. Further, in this MDP chapter, we assume the state space and action space are finite (in fact, typically small).

The following description of a simple machine as Markov decision process provides a concrete example of an MDP. The machine has three possible operations (*actions*): "wash", "paint", and "eject" (each with a corresponding button). Objects are put into the machine. Each time you push a button, something is done to the object. However, it's an old machine, so it's not very reliable. The machine has a camera inside that can clearly detect what is going on with the object and will output the state of the object: "dirty", "clean", "painted", or "ejected". For each action, this is what is done to the object:
Wash:

- If you perform the "wash" operation on any object, whether it's dirty, clean, or painted, it will end up "clean" with probability 0.9 and "dirty" otherwise.

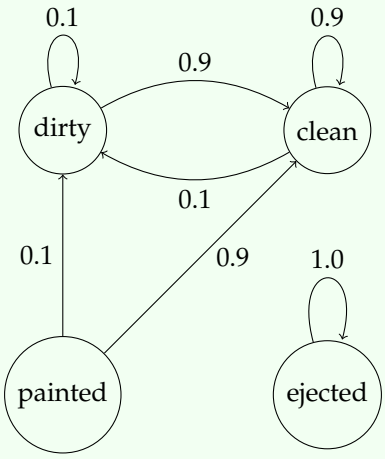
Paint:

- If you perform the "paint" operation on a clean object, it will become nicely "painted" with probability 0.8. With probability 0.1, the paint misses but the object stays clean, and also with probability 0.1, the machine dumps rusty dust all over the object and it becomes "dirty".
- If you perform the "paint" operation on a "painted" object, it stays "painted" with probability 1.0.
- If you perform the "paint" operation on a "dirty" part, it stays "dirty" with probability 1.0.

Eject:

- If you perform an "eject" operation on any part, the part comes out of the machine and this fun game is over. The part remains "ejected" regardless of any further action.

These descriptions specify the transition model T, and the transition function for each action can be depicted as a state machine diagram. For example, here is the diagram for "wash":



You get reward +10 for ejecting a painted object, reward 0 for ejecting a non-painted object, reward 0 for any action on an "ejected" object, and reward -3 otherwise. The MDP description would be completed by also specifying a discount factor.

A *policy* is a function $\pi : \mathcal{S} \to \mathcal{A}$ that specifies what action to take in each state. The policy is what we will want to learn; it is akin to the strategy that a player employs to win a given game. Below, we take just the initial steps towards this eventual goal. We describe how to evaluate how good a policy is, first in the *finite horizon* case (Section 10.1.1) when the total number of transition steps is finite. Then we consider the *infinite horizon* case (Section 10.1.2), when you don't know when the game will be over.

10.1.1 Finite-horizon value functions

The goal of a policy is to maximize the expected total reward, averaged over the stochastic transitions that the domain makes. Let's first consider the case where there is a finite *horizon* H, indicating the total number of steps of interaction that the agent will have with the MDP.

In the finite-horizon case, we usually set the discount factor γ to 1.

We seek to measure the goodness of a policy. We do so by defining for a given MDP policy π and horizon h, the "horizon h *value*" of a state, $V^h_\pi(s)$. We do this by induction on the horizon, which is the *number of steps left to go*.

The base case is when there are no steps remaining, in which case, no matter what state we're in, the value is 0, so

$$V^0_\pi(s) = 0 \ . \tag{10.1}$$

Then, the value of a policy in state s at horizon $h+1$ is equal to the reward it will get in state s plus the next state's expected horizon h value, discounted by a factor γ. So, starting with horizons 1 and 2, and then moving to the general case, we have:

$$V^1_\pi(s) = R(s, \pi(s)) + 0 \tag{10.2}$$

$$V^2_\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} T(s, \pi(s), s') V^1_\pi(s') \tag{10.3}$$

$$\vdots$$

$$V^h_\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} T(s, \pi(s), s') V^{h-1}_\pi(s') \tag{10.4}$$

The sum over s' is an *expectation*: it considers all possible next states s', and computes an average of their $(h-1)$-horizon values, weighted by the probability that the transition function from state s with the action chosen by the policy $\pi(s)$ assigns to arriving in state s', and discounted by γ.

Study Question: What is $\sum_{s'} T(s, a, s')$ for any particular s and a?

Study Question: Convince yourself that Eqs. 10.1 and 10.3 are special cases of Eq. 10.4.

Then we can say that a policy π_1 is better than policy π_2 for horizon h, i.e., $\pi_1 >_h \pi_2$, if and only if for all $s \in \mathcal{S}$, $V^h_{\pi_1}(s) \geqslant V^h_{\pi_2}(s)$ and there exists at least one $s \in \mathcal{S}$ such that $V^h_{\pi_1}(s) > V^h_{\pi_2}(s)$.

10.1.2 Infinite-horizon value functions

More typically, the actual finite horizon is not known, i.e., when you don't know when the game will be over! This is called the *infinite horizon* version of the problem. How does one evaluate the goodness of a policy in the infinite horizon case?

If we tried to simply take our definitions above and use them for an infinite horizon, we could get in trouble. Imagine we get a reward of 1 at each step under one policy and a reward of 2 at each step under a different policy. Then the reward as the number of steps grows in each case keeps growing to become infinite in the limit of more and more steps.

Even though it seems intuitive that the second policy should be better, we can't justify that by saying $\infty < \infty$.

One standard approach to deal with this problem is to consider the *discounted* infinite horizon. We will generalize from the finite-horizon case by adding a discount factor.

In the finite-horizon case, we valued a policy based on an expected finite-horizon value:

$$\mathbb{E}\left[\sum_{t=0}^{h-1} \gamma^t R_t \mid \pi, s_0\right] \quad , \tag{10.5}$$

where R_t is the reward received at time t.

What is $\mathbb{E}[\cdot]$? This mathematical notation indicates an *expectation*, i.e., an average taken over all the random possibilities which may occur for the argument. Here, the expectation is taken over the *conditional probability* $\Pr(R_t = r \mid \pi, s_0)$, where R_t is the random variable for the reward, subject to the policy being π and the state being s_0. Since π is a function, this notation is shorthand for conditioning on all of the random variables implied by policy π and the stochastic transitions of the MDP.

A very important point is that $R(s, a)$ is always deterministic (in this class) for any given s and a. Here R_t represents the set of all possible $R(s_t, a)$ at time step t; this R_t is a random variable because the state we're in at step t is itself a random variable, due to prior stochastic state transitions up to but not including at step t and prior (deterministic) actions dictated by policy π.

Now, for the infinite-horizon case, we select a discount factor $0 < \gamma < 1$, and evaluate a policy based on its expected *infinite horizon discounted value*:

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_t \mid \pi, s_0\right] = \mathbb{E}\left[R_0 + \gamma R_1 + \gamma^2 R_2 + \ldots \mid \pi, s_0\right] \quad . \tag{10.6}$$

Note that the t indices here are not the number of steps to go, but actually the number of steps forward from the starting state (there is no sensible notion of "steps to go" in the infinite horizon case).

Eqs. 10.5 and 10.6 are a conceptual stepping stone. Our main objective is to get to Eq. 10.8, which can also be viewed as including γ in Eq. 10.4, with the appropriate definition of the infinite-horizon value.

There are two good intuitive motivations for discounting. One is related to economic theory and the present value of money: you'd generally rather have some money today than that same amount of money next week (because you could use it now or invest it). The other is to think of the whole process terminating, with probability $1-\gamma$ on each step of the interaction. This value is the expected amount of reward the agent would gain under this terminating model.

At every step, your expected future lifetime, given that you have survived until now, is $1/(1-\gamma)$.

Study Question: Verify this fact: if, on every day you wake up, there is a probability of $1-\gamma$ that today will be your last day, then your expected lifetime is $1/(1-\gamma)$ days.

Let us now evaluate a policy in terms of the expected discounted infinite-horizon value that the agent will get in the MDP if it executes that policy. We define the infinite-horizon value of a state s under policy π as

$$V_\pi(s) = \mathbb{E}[R_0 + \gamma R_1 + \gamma^2 R_2 + \cdots \mid \pi, S_0 = s] = \mathbb{E}[R_0 + \gamma(R_1 + \gamma(R_2 + \gamma \ldots))) \mid \pi, S_0 = s] \quad . \tag{10.7}$$

Because the expectation of a linear combination of random variables is the linear combination of the expectations, we have

$$\begin{aligned} V_\pi(s) &= \mathbb{E}[R_0 \mid \pi, S_0 = s] + \gamma \mathbb{E}[R_1 + \gamma(R_2 + \gamma \dots))) \mid \pi, S_0 = s] \\ &= R(s, \pi(s)) + \gamma \sum_{s'} T(s, \pi(s), s') V_\pi(s') \ . \end{aligned} \tag{10.8}$$

The equation defined in Eq. 10.8 is known as the Bellman Equation, which breaks down the value function into the immediate reward and the (discounted) future value function. You could write down one of these equations for each of the $n = |\mathcal{S}|$ states. There are n unknowns $V_\pi(s)$. These are linear equations, and standard software (e.g., using Gaussian elimination or other linear algebraic methods) will, in most cases, enable us to find the value of each state under this policy.

This is *so* cool! In a discounted model, if you find that you survived this round and landed in some state s', then you have the same expected future lifetime as you did before. So the value function that is relevant in that state is exactly the same one as in state s.

10.2 Finding policies for MDPs

Given an MDP, our goal is typically to find a policy that is optimal in the sense that it gets as much total reward as possible, in expectation over the stochastic transitions that the domain makes. We build on what we have learned about evaluating the goodness of a policy (Sections 10.1.1 and 10.1.2), and find optimal policies for the finite horizon case (Section 10.2.1), then the infinite horizon case (Section 10.2.2).

10.2.1 Finding optimal finite-horizon policies

How can we go about finding an optimal policy for an MDP? We could imagine enumerating all possible policies and calculating their value functions as in the previous section and picking the best one – but that's too much work!

The first observation to make is that, in a finite-horizon problem, the best action to take depends on the current state, but also on the horizon: imagine that you are in a situation where you could reach a state with reward 5 in one step or a state with reward 100 in two steps. If you have at least two steps to go, then you'd move toward the reward 100 state, but if you only have one step left to go, you should go in the direction that will allow you to gain 5!

One way to find an optimal policy is to compute an optimal *action-value function*, Q. For the finite-horizon case, we define $Q^h(s, a)$ to be the expected value of

- starting in state s,
- executing action a, and
- continuing for $h-1$ more steps executing an optimal policy for the appropriate horizon on each step.

Similar to our definition of V^h for evaluating a policy, we define the Q^h function recursively according to the horizon. The only difference is that, on each step with horizon h, rather than selecting an action specified by a given policy, we select the value of a that will

maximize the expected Q^h value of the next state.

$$Q^0(s,a) = 0 \tag{10.9}$$

$$Q^1(s,a) = R(s,a) + 0 \tag{10.10}$$

$$Q^2(s,a) = R(s,a) + \gamma \sum_{s'} T(s,a,s') \max_{a'} Q^1(s',a') \tag{10.11}$$

$$\vdots$$

$$Q^h(s,a) = R(s,a) + \gamma \sum_{s'} T(s,a,s') \max_{a'} Q^{h-1}(s',a') \tag{10.12}$$

where (s',a') denotes the next time-step state/action pair. We can solve for the values of Q^h with a simple recursive algorithm called *finite-horizon value iteration* that just computes Q^h starting from horizon 0 and working backward to the desired horizon H. Given Q^h, an optimal π^*_h can be found as follows:

$$\pi^*_h(s) = \arg\max_a Q^h(s,a) \quad . \tag{10.13}$$

which gives the *immediate* best action(s) to take when there are h steps left; then $\pi^*_{h-1}(s)$ gives the best action(s) when there are $(h-1)$ steps left, and so on. In the case where there are multiple best actions, we typically can break ties randomly.

Additionally, it is worth noting that in order for such an optimal policy to be computed, we assume that the reward function $R(s,a)$ is bounded on the set of all possible (state, action) pairs. Furthermore, we will assume that the set of all possible actions is finite.

Study Question: The optimal value function is unique, but the optimal policy is not. Think of a situation in which there is more than one optimal policy.

Dynamic programming (somewhat counter-intuitively, dynamic programming is neither really "dynamic" nor a type of "programming" as we typically understand it) is a technique for designing efficient algorithms. Most methods for solving MDPs or computing value functions rely on dynamic programming to be efficient. The *principle of dynamic programming* is to compute and store the solutions to simple sub-problems that can be re-used later in the computation. It is a very important tool in our algorithmic toolbox.
Let's consider what would happen if we tried to compute $Q^4(s, a)$ for all (s, a) by directly using the definition:

- To compute $Q^4(s_i, a_j)$ for any one (s_i, a_j), we would need to compute $Q^3(s, a)$ for all (s, a) pairs.
- To compute $Q^3(s_i, a_j)$ for any one (s_i, a_j), we'd need to compute $Q^2(s, a)$ for all (s, a) pairs.
- To compute $Q^2(s_i, a_j)$ for any one (s_i, a_j), we'd need to compute $Q^1(s, a)$ for all (s, a) pairs.
- Luckily, those are just our $R(s, a)$ values.

So, if we have n states and m actions, this is $O((mn)^3)$ work — that seems like way too much, especially as the horizon increases! But observe that we really only have mnh values that need to be computed: $Q^h(s, a)$ for all h, s, a. If we start with $h = 1$, compute and store those values, then using and reusing the $Q^{h-1}(s, a)$ values to compute the $Q^h(s, a)$ values, we can do all this computation in time $O(mn^2h)$, which is much better!

10.2.2 Finding optimal infinite-horizon policies

In contrast to the finite-horizon case, the best way of behaving in an infinite-horizon discounted MDP is not time-dependent. That is, the decisions you make at time $t = 0$ looking forward to infinity, will be the same decisions that you make at time $t = T$ for any positive T, also looking forward to infinity.

An important theorem about MDPs is: in the infinite-horizon case, there exists a stationary optimal policy π^* (there may be more than one) such that for all $s \in \mathcal{S}$ and all other policies π, we have

$$V_{\pi^*}(s) \geqslant V_{\pi}(s) \quad . \tag{10.14}$$

Stationary means that it doesn't change over time; in contrast, the optimal policy in a finite-horizon MDP is *non-stationary.*

There are many methods for finding an optimal policy for an MDP. We have already seen the finite-horizon value iteration case. Here we will study a very popular and useful method for the infinite-horizon case, *infinite-horizon value iteration*. It is also important to us, because it is the basis of many *reinforcement-learning* methods.

We will again assume that the reward function $R(s, a)$ is bounded on the set of all possible (state, action) pairs and additionally that the number of actions in the action space is finite. Define $Q(s, a)$ to be the expected infinite-horizon discounted value of being in state s, executing action a, and executing an optimal policy π^* thereafter. Using similar reasoning to the recursive definition of V_π, we can express this value recursively as

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') \max_{a'} Q(s', a') \quad . \tag{10.15}$$

This is also a set of equations, one for each (s, a) pair. This time, though, they are not linear (due to the max operation), and so they are not easy to solve. But there is a theorem

that says they have a unique solution!

Once we know the optimal action-value function, then we can extract an optimal policy π^* as

$$\pi^*(s) = \arg\max_a Q(s,a) \quad . \tag{10.16}$$

As in the finite-horizon case, there may be more than one optimal policy in the infinite-horizon case.

We can iteratively solve for the Q^* values with the infinite-horizon value iteration algorithm, shown below:

INFINITE-HORIZON-VALUE-ITERATION$(\mathcal{S}, \mathcal{A}, T, R, \gamma, \epsilon)$

```
for s ∈ S, a ∈ A :
    Q_old(s, a) = 0
while not converged:
    for s ∈ S, a ∈ A :
        Q_new(s, a) = R(s, a) + γ Σ_s' T(s, a, s') max_a' Q_old(s', a')
    if max_{s,a} |Q_old(s, a) − Q_new(s, a)| < ε :
        return Q_new
    Q_old = Q_new
```

Theory There are a lot of nice theoretical results about infinite-horizon value iteration. For some given (not necessarily optimal) Q function, define $\pi_Q(s) = \arg\max_a Q(s,a)$.

- After executing infinite-horizon value iteration with convergence hyper-parameter ϵ,

$$\|V_{\pi_{Q_{new}}} - V_{\pi^*}\|_{max} < \epsilon \quad . \tag{10.17}$$

Note the new notation! Given two functions f and f', we write $\|f - f'\|_{max}$ to mean $\max_x |f(x) - f'(x)|$. It measures the maximum absolute disagreement between the two functions at any input x.

- There is a value of ϵ such that

$$\|Q_{old} - Q_{new}\|_{max} < \epsilon \implies \pi_{Q_{new}} = \pi^* \tag{10.18}$$

- As the algorithm executes, $\|V_{\pi_{Q_{new}}} - V_{\pi^*}\|_{max}$ decreases monotonically on each iteration.
- The algorithm can be executed asynchronously, in parallel: as long as all (s,a) pairs are updated infinitely often in an infinite run, it still converges to the optimal value.

This is very important for reinforcement learning.

CHAPTER 11

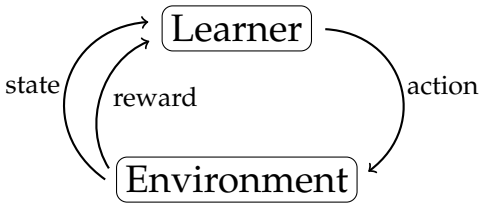
Reinforcement learning

So far, all the learning problems we have looked at have been *supervised*, that is, for each training input $x^{(i)}$, we are told which value $y^{(i)}$ should be the output. *Reinforcement learning* differs from previous learning problems in several important ways:

- The learner interacts explicitly with an environment, rather than implicitly (as in supervised learning) through an available training data set of $(x^{(i)}, y^{(i)})$ pairs drawn from the environment.
- The learner has some choice over what new information it seeks to gain from the environment.
- The learner updates models incrementally as additional information about the environment becomes available.

Online learning is a variant of supervised learning in which new data pairs become available over time and the model is updated, e.g., by retraining over the entire larger data set, or by weight update using just the new data.

In a reinforcement learning problem, the interaction with the environment takes a particular form:



- Learner observes *input* state $s^{(i)}$
- Learner generates *output* action $a^{(i)}$
- Learner observes *reward* $r^{(i)}$
- Learner observes *input* state $s^{(i+1)}$
- Learner generates *output* action $a^{(i+1)}$
- Learner observes *reward* $r^{(i+1)}$
- ...

Similar to MDPs, the learner is supposed to find a *policy*, mapping a state s to action a, that maximizes expected reward over time.

11.1 Reinforcement learning algorithms overview

A *reinforcement learning (*RL*) algorithm* is a kind of a policy that depends on the whole history of states, actions, and rewards and selects the next action to take. There are several different ways to measure the quality of an RL algorithm, including:

- Ignoring the $r^{(i)}$ values that it gets *while* learning, but considering how many interactions with the environment are required for it to learn a policy $\pi : \mathcal{S} \to \mathcal{A}$ that is nearly optimal.
- Maximizing the expected sum of discounted rewards while it is learning.

Most of the focus is on the first criterion (which is called "sample efficiency"), because the second one is very difficult. The first criterion is reasonable when the learning can take place somewhere safe (imagine a robot learning, inside the robot factory, where it can't hurt itself too badly) or in a simulated environment.

Approaches to reinforcement learning differ significantly according to what kind of hypothesis or model is being learned. Roughly speaking, RL methods can be categorized into model-free methods and model-based methods. The main distinction is that model-based methods explicitly learn the transition and reward models to assist the end-goal of learning a policy; model-free methods do not. We will start our discussion with the model-free methods, and introduce two of the arguably most popular types of algorithms, Q-learning (Section 11.2.1) and policy gradient (Section 11.2.4). We then describe model-based methods (Section 11.3). Finally, we briefly consider "bandit" problems (Section 11.4), which differ from our MDP learning context by having probabilistic rewards.

11.2 Model-free methods

Model-free methods are methods that do not explicitly learn transition and rewards models. Depending on what is explicitly being learned, model-free methods are sometimes further categorized into value-based methods (where the goal is to learn/estimate a value function) and policy-based methods (where the goal is to directly learn an optimal policy). It's important to note that such categorization is approximate and the boundaries are blurry. In fact, current RL research tends to combine the learning of value functions, policies, and transition and reward models all into a complex learning algorithm, in an attempt to combine the strengths of each approach.

11.2.1 Q-learning

Q-learning is a frequently used class of RL algorithms that concentrates on learning (estimating) the state-action value function, i.e., the Q function. Specifically, recall the MDP value-iteration update:

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') \max_{a'} Q(s', a') \tag{11.1}$$

The thing that most students seem to get confused about is when we do value iteration and when we do Q learning. Value iteration assumes you know T and R and just need to *compute* Q. In Q learning, we don't know or even directly estimate T and R: we estimate Q directly from experience!

The Q-learning algorithm below adapts this value-iteration idea to the RL scenario, where we do not know the transition function T or reward function R, and instead rely on samples to perform the updates.

Q-LEARNING($\mathcal{S}, \mathcal{A}, \gamma, \epsilon, \alpha, s_0$)

```
for s ∈ S, a ∈ A :
    Q_old(s, a) = 0
s = s_0
while True:
    a = select_action(s, Q_old(s, a))
    r, s' = execute(a)
    Q_new(s, a) = (1 − α)Q_old(s, a) + α(r + γ max_a' Q_old(s', a'))
    s = s'
    if max_{s,a} |Q_old(s, a) − Q_new(s, a)| < ε :
        return Q_new
    Q_old = Q_new
```

With the pseudo-code provided for Q-learning, there are a few key things to note. First, we must determine which state to initialize the learning from. In the context of a game, this initial state may be well defined. In the context of a robot navigating an environment, one may consider sampling the initial state at random. In any case, the initial state is necessary to determine the trajectory the agent will experience as it navigates the environment. Second, different contexts will influence how we want to choose when to stop iterating through the while loop. Again, in some games there may be a clear terminating state based on the rules of how it is played. On the other hand, a robot may be allowed to explore an environment *ad infinitum*. In such a case, one may consider either setting a fixed number of transitions to take or we may want to stop iterating once the values in the Q-table are not changing. Finally, a single trajectory through the environment may not be sufficient to adequately explore all state-action pairs. In these instances, it becomes necessary to run through a number of iterations of the Q-learning algorithm, potentially with different choices of initial state s_0. Of course, we would then want to modify Q-learning such that the Q table is not reset with each call.

This notion of running a number of instances of Q-learning is often referred to as experiencing multiple *episodes*.

Now, let's dig in to what is happening in Q-learning. Here, $\alpha \in (0, 1]$ represents the "learning rate," which needs to decay for convergence purposes, but in practice is often set to a constant. It's also worth mentioning that Q-learning assumes a discrete state and action space where states and actions take on discrete values like $1, 2, 3, \ldots$ etc. In contrast, a continuous state space would allow the state to take values from, say, a continuous range of numbers; for example, the state could be any real number in the interval $[1, 3]$. Similarly, a continuous action space would allow the action to be drawn from, e.g., a continuous range of numbers. There are now many extensions developed based on Q-learning that can handle continuous state and action spaces (we'll look at one soon), and therefore the algorithm above is also sometimes referred to more specifically as tabular Q-learning.

In the Q-learning update rule

$$Q[s, a] \leftarrow (1 - \alpha)Q[s, a] + \alpha(r + \gamma \max_{a'} Q[s', a']) \tag{11.2}$$

the term $r + \gamma \max_{a'} Q[s', a']$ is often referred to as the (one-step look-ahead) *target*. The update can be viewed as a combination of two different iterative processes that we have already seen: the combination of an old estimate with the target using a running average with a learning rate α, and the dynamic-programming update of a Q value from value iteration.

Eq. 11.2 can also be equivalently rewritten as

$$Q[s, a] \leftarrow Q[s, a] + \alpha \left((r + \gamma \max_{a'} Q[s', a']) - Q[s, a] \right), \tag{11.3}$$

which allows us to interpret Q-learning in yet another way: we make an update (or correction) based on the temporal difference between the target and the current estimated value

$Q[s, a]$.

The Q-learning algorithm above includes a procedure called *select_action*, that, given the current state s and current Q function, has to decide which action to take. If the Q value is estimated very accurately and the agent is behaving in the world, then generally we would want to choose the apparently optimal action $\arg\max_{a \in \mathcal{A}} Q(s, a)$. But, during learning, the Q value estimates won't be very good and exploration is important. However, exploring completely at random is also usually not the best strategy while learning, because it is good to focus your attention on the parts of the state space that are likely to be visited when executing a good policy (not a bad or random one).

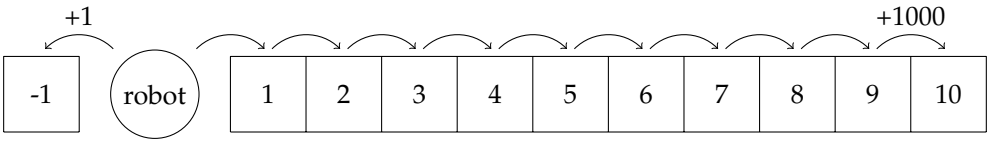
A typical action-selection strategy that attempts to address this *exploration versus exploitation* dilemma is the so-called ϵ-*greedy* strategy:

- with probability $1 - \epsilon$, choose $\arg\max_{a \in \mathcal{A}} Q(s, a)$;
- with probability ϵ, choose the action $a \in \mathcal{A}$ uniformly at random.

where the ϵ probability of choosing a random action helps the agent to explore and try out actions that might not seem so desirable at the moment.

Q-learning has the surprising property that it is *guaranteed* to converge to the actual optimal Q function under fairly weak conditions! Any exploration strategy is okay as long as it tries every action infinitely often on an infinite run (so that it doesn't converge prematurely to a bad action choice).

Q-learning can be very inefficient. Imagine a robot that has a choice between moving to the left and getting a reward of 1, then returning to its initial state, or moving to the right and walking down a 10-step hallway in order to get a reward of 1000, then returning to its initial state.



The first time the robot moves to the right and goes down the hallway, it will update the Q value just for state 9 on the hallway and action "right" to have a high value, but it won't yet understand that moving to the right in the earlier steps was a good choice. The next time it moves down the hallway it updates the value of the state before the last one, and so on. After 10 trips down the hallway, it now can see that it is better to move to the right than to the left.

More concretely, consider the vector of Q values $Q(i = 0, \ldots, 9; \text{right})$, representing the Q values for moving right at each of the positions $i = 0, \ldots, 9$. Position index 0 is the starting position of the robot as pictured above.

Then, for $\alpha = 1$ and $\gamma = 0.9$, Eq. 11.3 becomes

$$Q(i, \text{ right}) = R(i, \text{ right}) + 0.9 \cdot \max_a Q(i + 1, a). \tag{11.4}$$

Starting with Q values of 0,

$$Q^{(0)}(i = 0, \ldots, 9; \text{ right}) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}. \tag{11.5}$$

Since the only nonzero reward from moving right is $R(9, \text{ right}) = 1000$, after our robot makes it down the hallway once, our new Q vector is

$$Q^{(1)}(i = 0, \ldots, 9; \text{ right}) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1000 \end{bmatrix}. \tag{11.6}$$

We are violating our usual notational conventions here, and writing $Q^{(i)}$ to mean the Q value function that results after the robot runs all the way to the end of the hallway, when executing the policy that always moves to the right.

After making its way down the hallway again, $Q(8, \text{right}) = 0 + 0.9 \cdot Q(9, \text{right}) = 900$ updates:

$$Q^{(2)}(i = 0, \ldots, 9; \text{right}) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 900 & 1000 \end{bmatrix}. \tag{11.7}$$

Similarly,

$$Q^{(3)}(i = 0, \ldots, 9; \text{right}) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 810 & 900 & 1000 \end{bmatrix} \tag{11.8}$$

$$Q^{(4)}(i = 0, \ldots, 9; \text{right}) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 729 & 810 & 900 & 1000 \end{bmatrix} \tag{11.9}$$

$$\vdots \tag{11.10}$$

$$Q^{(10)}(i = 0, \ldots, 9; \text{right}) = \begin{bmatrix} 387.4 & 420.5 & 478.3 & 531.4 & 590.5 & 656.1 & 729 & 810 & 900 & 1000 \end{bmatrix}, \tag{11.11}$$

and the robot finally sees the value of moving right from position 0.

Here, we can see the exploration/exploitation dilemma in action: from the perspective of $s_0 = 0$, it will seem that getting the immediate reward of 1 is a better strategy without exploring the long hallway.

Study Question: Determine the Q value functions that will result from updates due to the robot always executing the "move left" policy.

11.2.2 Function approximation: Deep Q learning

In our Q-learning algorithm above, we essentially keep track of each Q value in a table, indexed by s and a. What do we do if $\mathcal{S}$ and/or $\mathcal{A}$ are large (or continuous)?

We can use a function approximator like a neural network to store Q values. For example, we could design a neural network that takes in inputs s and a, and outputs $Q(s, a)$. We can treat this as a regression problem, optimizing this loss:

$$\left(Q(s, a) - (r + \gamma \max_{a'} Q(s', a')) \right)^2, \tag{11.12}$$

This is the so-called squared Bellman error; as the name suggests, it's closely related to the Bellman equality we saw in MDPs in Chapter 10. Roughly speaking, this error measures how much the Bellman equality is violated.

where $Q(s, a)$ is now the output of the neural network.

There are several different architectural choices for using a neural network to approximate Q values:

- One network for each action a, that takes s as input and produces $Q(s, a)$ as output;
- One single network that takes s as input and produces a vector $Q(s, \cdot)$, consisting of the Q values for each action; or
- One single network that takes s, a concatenated into a vector (if a is discrete, we would probably use a one-hot encoding, unless it had some useful internal structure) and produces $Q(s, a)$ as output.

For continuous action spaces, it is popular to use a class of methods called *actor-critic* methods, which combine policy and value-function learning. We won't get into them in detail here, though.

The first two choices are only suitable for discrete (and not too big) action sets. The last choice can be applied for continuous actions, but then it is difficult to find $\arg\max_{a \in \mathcal{A}} Q(s, a)$.

There are not many theoretical guarantees about Q-learning with function approximation and, indeed, it can sometimes be fairly unstable (learning to perform well for a while, and then getting suddenly worse, for example). But neural network Q-learning has also had some significant successes.

One form of instability that we do know how to guard against is *catastrophic forgetting*. In standard supervised learning, we expect that the training x values were drawn independently from some distribution. But when a learning agent, such as a robot, is moving through an environment, the sequence of states it encounters will be temporally correlated. For example, the robot might spend 12 hours in a dark environment and then 12 in a light

And, in fact, we routinely shuffle their order in the data file, anyway.

one. This can mean that while it is in the dark, the neural-network weight-updates will make the Q function "forget" the value function for when it's light.

One way to handle this is to use *experience replay*, where we save our (s, a, s', r) experiences in a *replay buffer*. Whenever we take a step in the world, we add the (s, a, s', r) to the replay buffer and use it to do a Q-learning update. Then we also randomly select some number of tuples from the replay buffer, and do Q-learning updates based on them, as well. In general it may help to keep a *sliding window* of just the 1000 most recent experiences in the replay buffer. (A larger buffer will be necessary for situations when the optimal policy might visit a large part of the state space, but we like to keep the buffer size small for memory reasons and also so that we don't focus on parts of the state space that are irrelevant for the optimal policy.) The idea is that it will help us propagate reward values through our state space more efficiently if we do these updates. We can see it as doing something like value iteration, but using samples of experience rather than a known model.

11.2.3 Fitted Q-learning

An alternative strategy for learning the Q function that is somewhat more robust than the standard Q-learning algorithm is a method called *fitted Q*.

FITTED-Q-LEARNING($\mathcal{A}, s_0, \gamma, \alpha, \epsilon, m$)

```
1  s = s_0 // (e.g., s_0 can be drawn randomly from S)
2  D = { }
3  initialize neural-network representation of Q
4  while True:
5      D_new = experience from executing ε-greedy policy based on Q for m steps
6      D = D ∪ D_new represented as (s, a, s', r) tuples
7      D_supervised = {(x^(i), y^(i))} where x^(i) = (s, a) and y^(i) = r + γ max_{a'∈A} Q(s', a')
8        for each tuple (s, a, s', r)^(i) ∈ D
9      re-initialize neural-network representation of Q
10     Q = SUPERVISED-NN-REGRESSION(D_supervised)
```

Here, we alternate between using the policy induced by the current Q function to gather a batch of data $\mathcal{D}_{\text{new}}$, adding it to our overall data set $\mathcal{D}$, and then using supervised neural-network training to learn a representation of the Q value function on the whole data set. This method does not mix the dynamic-programming phase (computing new Q values based on old ones) with the function approximation phase (supervised training of the neural network) and avoids catastrophic forgetting. The regression training in line 9 typically uses squared error as a loss function and would be trained until the fit is good (possibly measured on held-out data).

11.2.4 Policy gradient

A different model-free strategy is to search directly for a good policy. The strategy here is to define a functional form $f(s; \theta) = a$ for the policy, where θ represents the parameters we learn from experience. We choose f to be differentiable, and often define $f(s, a; \theta) = \Pr(a|s)$, a conditional probability distribution over our possible actions.

This means the chance of choosing an action depends on which state the agent is in. Suppose, e.g., a robot is trying to get to a goal and can go left or right. An unconditional policy can say: I go left 99% of the time; a conditional policy can consider the robot's state, and say: if I'm to the right of the goal, I go left 99% of the time.

Now, we can train the policy parameters using gradient descent:

- When θ has relatively low dimension, we can compute a numeric estimate of the gradient by running the policy multiple times for different values of θ, and computing the resulting rewards.

- When θ has higher dimensions (e.g., it represents the set of parameters in a complicated neural network), there are more clever algorithms, e.g., one called REINFORCE, but they can often be difficult to get to work reliably.

Policy search is a good choice when the policy has a simple known form, but the MDP would be much more complicated to estimate.

11.3 Model-based RL

The conceptually simplest approach to RL is to model R and T from the data we have gotten so far, and then use those models, together with an algorithm for solving MDPs (such as value iteration) to find a policy that is near-optimal given the current models.

Assume that we have had some set of interactions with the environment, which can be characterized as a set of tuples of the form $(s^{(t)}, a^{(t)}, s^{(t+1)}, r^{(t)})$.

Because the transition function $T(s, a, s')$ specifies probabilities, multiple observations of (s, a, s') may be needed to model the transition function. One approach to this task of building a model $\hat{T}(s, a, s')$ for the true $T(s, a, s')$ is to estimate it using a simple counting strategy,

$$\hat{T}(s, a, s') = \frac{\#(s, a, s') + 1}{\#(s, a) + |\mathcal{S}|}. \tag{11.13}$$

Here, $\#(s, a, s')$ represents the number of times in our data set we have the situation where $s^{(t)} = s, a^{(t)} = a, s^{(t+1)} = s'$ and $\#(s, a)$ represents the number of times in our data set we have the situation where $s^{(t)} = s, a^{(t)} = a$.

Study Question: Prove to yourself that $\#(s, a) = \sum_{s'} \#(s, a, s')$.

Adding 1 and $|\mathcal{S}|$ to the numerator and denominator, respectively, are a form of smoothing called the *Laplace correction*. It ensures that we never estimate that a probability is 0, and keeps us from dividing by 0. As the amount of data we gather increases, the influence of this correction fades away.

Conceptually, this is also similar to having "initialized" our estimate for the transition function with uniform random probabilities, before having made any observations.

In contrast, the reward function $R(s, a)$ (as we have specified it in this text) is a *deterministic* function, such that knowing the reward r for a given (s, a) is sufficient to fully determine the function at that point. In other words, our model $\hat{R}$ can simply be a record of observed rewards, such that $\hat{R}(s, a) = r = R(s, a)$.

Given empirical models $\hat{T}$ and $\hat{R}$ for the transition and reward functions, we can now solve the MDP $(\mathcal{S}, \mathcal{A}, \hat{T}, \hat{R})$ to find an optimal policy using value iteration, or use a search algorithm to find an action to take for a particular state.

This approach is effective for problems with small state and action spaces, where it is not too hard to get enough experience to model T and R well; but it is difficult to generalize this method to handle continuous (or very large discrete) state spaces, and is a topic of current research.

11.4 Bandit problems

Bandit problems differ from our reinforcement learning setting as described above in two ways: the reward function is probabilistic, and the key decision is usually framed as whether or not to continue exploring (to improve the model) versus exploiting (take actions to maximize expected rewards based on the current model).

A basic bandit problem is given by

- A set of actions $\mathcal{A}$;

- A set of reward values $\mathcal{R}$; and
- A probabilistic reward function $R_p : \mathcal{A} \times \mathcal{R} \to \mathbb{R}$, i.e., R_p is a function that takes an action and a reward and returns the probability of getting that reward conditioned on that action being taken, $R_p(a, r) = \Pr(\text{reward} = r \mid \text{action} = a)$. This is analogous to how the transition function T is defined. Each time the agent takes an action, a new value is drawn from this distribution.

The most typical bandit problem has $\mathcal{R} = \{0, 1\}$ and $|\mathcal{A}| = k$. This is called a k-*armed bandit problem*, where the decision is which "arm" (action a) to select, and the reward is either getting a payoff (1) or not (0). There is a lot of mathematical literature on optimal strategies for k-armed bandit problems under various assumptions. The important question is usually one of *exploration versus exploitation*. Imagine that you have tried each action 10 times, and now you have estimates $\hat{R}_p(a, r)$ for the probabilities $R_p(a, r)$ for reward r given action a. Which arm should you pick next? You could

Why? Because in English slang, "one-armed bandit" is a name for a slot machine (an old-style gambling machine where you put a coin into a slot and then pull its arm to see if you get a payoff) because it has one arm and takes your money! What we have here is a similar sort of machine, but with k arms.

exploit your knowledge, and for future trials choose the arm with the highest value of expected reward; or

explore further, by trying some or all actions more times, hoping to get better estimates of the $R_p(a, r)$ values.

The theory ultimately tells us that, the longer our horizon h (or, similarly, closer to 1 our discount factor), the more time we should spend exploring, so that we don't converge prematurely on a bad choice of action.

Study Question: Why is it that "bad" luck during exploration is more dangerous than "good" luck? Imagine that there is an action that generates reward value 1 with probability 0.9, but the first three times you try it, it generates value 0. How might that cause difficulty? Why is this more dangerous than the situation when an action that generates reward value 1 with probability 0.1 actually generates reward 1 on the first three tries?

Bandit problems are reinforcement learning problems (and are very different from batch supervised learning) in that:

There is a setting of supervised learning, called *active learning*, where instead of being given a training set, the learner gets to select a value of x and the environment gives back a label y; the problem of picking good x values to query is interesting, but the problem of deriving a hypothesis from (x, y) pairs is the same as the supervised problem we have been studying.

- The agent gets to influence what data it obtains (selecting a gives it another sample from $R(a, r)$), and
- The agent is penalized for mistakes it makes while it is learning (if it is trying to maximize the expected reward $\sum_r r \cdot \Pr(R_p(a, r) = r)$ it gets while behaving).

In a *contextual* bandit problem, you have multiple possible states, drawn from some set S, and a separate bandit problem associated with each one.

Bandit problems are an essential subset of reinforcement learning. It's important to be aware of the issues, but we will not study solutions to them in this class.

CHAPTER 12

Unsupervised Learning

In previous chapters, we have largely focused on classification and regression problems, where we use supervised learning with training samples that have both features/inputs and corresponding outputs or labels, to learn hypotheses or models that can then be used to predict labels for new data.

In contrast to supervised learning paradigm, we can also have an unsupervised learning setting, where we only have features but no corresponding outputs or labels for our dataset. On natural question aries then: if there are no labels, what are we learning?

One canonical example of unsupervised learning is *clustering*, which we will learn about in Section 12.1. In clustering, the goal is to develop algorithms that can reason about "similarity" among data points's features, and group the data points into clusters.

Autoencoders are another family of unsupervised learning algorithms, which we will look at in Section 12.2, and in this case, we will be seeking to obtain insights about our data by learning compressed versions of the original data. Or, in other words, by finding a good lower-dimensional feature representations of the same data set. Such insights might help us to discover and characterize underlying factors of variation in data, which can aid in scientific discovery; to compress data for efficient storage or communication; or to preprocess our data prior to supervised learning, perhaps to reduce the amount of data that is needed to learn a good classifier or regressor.

12.1 Clustering

Oftentimes a dataset can be partitioned into different categories. A doctor may notice that their patients come in cohorts and different cohorts respond to different treatments. A biologist may gain insight by identifying that bats and whales, despite outward appearances, have some underlying similarity, and both should be considered members of the same category, i.e., "mammal". The problem of automatically identifying meaningful groupings in datasets is called clustering. Once these groupings are found, they can be leveraged toward interpreting the data and making optimal decisions for each group.

12.1.1 Clustering formalisms

Mathematically, clustering looks a bit like classification: we wish to find a mapping from datapoints, x, to categories, y. However, rather than the categories being predefined labels, the categories in clustering are automatically discovered *partitions* of an unlabeled dataset.

Because clustering does not learn from labeled examples, it is an example of an *unsupervised* learning algorithm. Instead of mimicking the mapping implicit in supervised training pairs $\{x^{(i)}, y^{(i)}\}_{i=1}^{n}$, clustering assigns datapoints to categories based on how the unlabeled data $\{x^{(i)}\}_{i=1}^{n}$ is *distributed* in data space.

Intuitively, a "cluster" is a group of datapoints that are all nearby to each other and far away from other clusters. Let's consider the following scatter plot. How many clusters do you think there are?

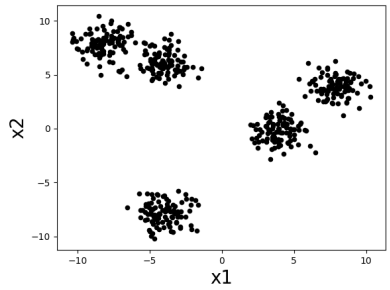


Figure 12.1: A dataset we would like to cluster. How many clusters do you think there are?

There seem to be about five clumps of datapoints and those clumps are what we would like to call clusters. If we assign all datapoints in each clump to a cluster corresponding to that clump, then we might desire that nearby datapoints are assigned to the same cluster, while far apart datapoints are assigned to different clusters.

In designing clustering algorithms, three critical things we need to decide are:

- How do we measure *distance* between datapoints? What counts as "nearby" and "far apart"?
- How many clusters should we look for?
- How do we evaluate how good a clustering is?

We will see how to begin making these decisions as we work through a concrete clustering algorithm in the next section.

12.1.2 The k-means formulation

One of the simplest and most commonly used clustering algorithms is called k-means. The goal of the k-means algorithm is to assign datapoints to k clusters in such a way that the variance within clusters is as small as possible. Notice that this matches our intuitive idea that a cluster should be a tightly packed set of datapoints.

Similar to the way we showed that supervised learning could be formalized mathematically as the minimization of an objective function (loss function + regularization), we will show how unsupervised learning can also be formalized as minimizing an objective function. Let us denote the cluster assignment for a datapoint $x^{(i)}$ as $y^{(i)} \in \{1, 2, \ldots, k\}$, i.e., $y^{(i)} = 1$ means we are assigning datapoint $x^{(i)}$ to cluster number 1. Then the k-means

We will be careful to distinguish between the k-means *algorithm* and the k-means *objective*. As we will see, the k-means algorithm can be understood to be just one optimization algorithm which finds a local optimum of the k-means objective.

Recall that *variance* is a measure of how "spread out" data is, defined as the mean squared distance from the average value of the data.

objective can be quantified with the following objective function (which we also call the "k-means objective" or "k-means loss"):

$$\sum_{j=1}^{k} \sum_{i=1}^{n} \mathbb{1}(y^{(i)} = j) \left\| x^{(i)} - \mu^{(j)} \right\|^2 , \tag{12.1}$$

where $\mu^{(j)} = \frac{1}{N_j} \sum_{i=1}^{n} \mathbb{1}(y^{(i)} = j) x^{(i)}$ and $N_j = \sum_{i=1}^{n} \mathbb{1}(y^{(i)} = j)$, so that $\mu^{(j)}$ is the mean of all datapoints in cluster j, and using $\mathbb{1}(\cdot)$ to denote the indicator function (which takes on value of 1 if its argument is true and 0 otherwise). The inner sum (over data points) of the loss is the variance of datapoints within cluster j. We sum up the variance of all k clusters to get our overall loss.

12.1.3 K-means algorithm

The k-means algorithm minimizes this loss by alternating between two steps: given some initial cluster assignments: 1) compute the mean of all data in each cluster and assign this as the "cluster mean", and 2) reassign each datapoint to the cluster with nearest cluster mean. Fig. 12.2 shows what happens when we repeat these steps on the dataset from above.

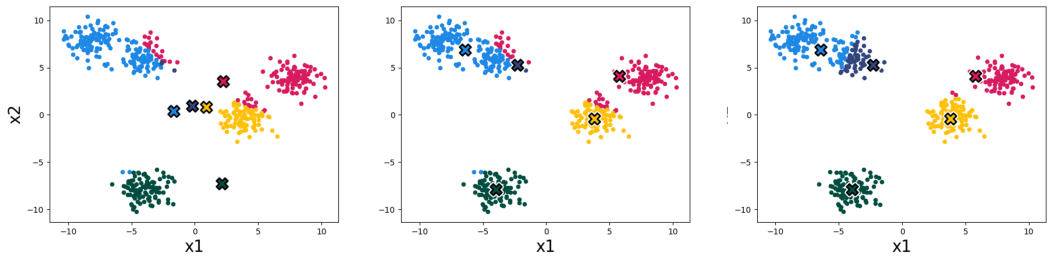


Figure 12.2: The first three steps of running the k-means algorithm on this data. Datapoints are colored according to the cluster to which they are assigned. Cluster means are the larger X's with black outlines.

Each time we reassign the data to the nearest cluster mean, the k-means loss decreases (the datapoints end up closer to their assigned cluster mean), or stays the same. And each time we recompute the cluster means the loss *also* decreases (the means end up closer to their assigned datapoints) or stays the same. Overall then, the clustering gets better and better, according to our objective – until it stops improving. After four iterations of cluster assignment + update means in our example, the k-means algorithm stops improving. Its final solution is shown in Fig. 12.3. It seems to arrive at something reasonable!

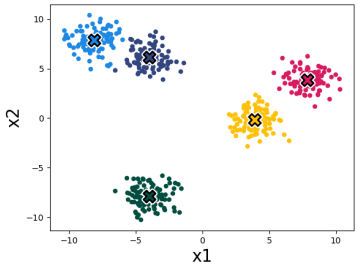


Figure 12.3: K-means loss stops improving; final result.

Now let's write out the algorithm in complete detail:

K-MEANS$(k, \tau, \{x^{(i)}\}_{i=1}^{n})$

```
1  μ, y = random initialization
2  for t = 1 to τ
3      y_old = y
4      for i = 1 to n
5          y^(i) = arg min_j ||x^(i) − μ^(j)||^2
6      for j = 1 to k
7          μ^(j) = (1/N_j) Σ_{i=1}^{n} 𝟙(y^(i) = j) x^(i)
8      if 𝟙(y = y_old)
9          break
10 return μ, y
```

Study Question: Why do we have the "break" statement on line 9? Could the clustering improve if we ran it for more iterations after this point? Has it converged?

The for-loop over the n datapoints assigns each datapoint to the nearest cluster center. The for-loop over the k clusters updates the cluster center to be the mean of all datapoints currently assigned to that cluster. As suggested above, it can be shown that this algorithm reduces the loss in Eq. 12.1 on each iteration, until it converges to a local minimum of the loss.

It's like classification except the algorithm *picked* what the classes are rather than being given examples of what the classes are.

12.1.4 Using gradient descent to minimize k-means objective

We can also use gradient descent to optimize the k-means objective. To show how to apply gradient descent, we first rewrite the objective as a differentiable function *only of* μ:

$$L(\mu) = \sum_{i=1}^{n} \min_j \left\| x^{(i)} - \mu^{(j)} \right\|^2 \quad . \tag{12.2}$$

The k-means algorithm presented above is a form of *block coordinate descent*, rather than gradient descent. For certain problems, and in particular k-means, this method can converge faster than gradient descent.

$L(\mu)$ is the value of the k-means loss given that we pick the *optimal* assignments of the datapoints to cluster means (that's what the $\min_j$ does). Now we can use the gradient $\frac{\partial L(\mu)}{\partial \mu}$ to find the values for μ that achieve minimum loss when cluster assignments are optimal. Finally, we read off the optimal cluster assignments, given the optimized μ, just by assigning datapoints to their nearest cluster mean:

$$y^{(i)} = \arg\min_j \left\| x^{(i)} - \mu^{(j)} \right\|^2 \quad . \tag{12.3}$$

$L(\mu)$ is a smooth function except with kinks where the nearest cluster changes; that means it's differentiable almost everywhere, which in practice is sufficient for us to apply gradient descent.

This procedure yields a local minimum of Eq. 12.1, as does the standard k-means algorithm we presented (though they might arrive at different solutions). It might not be a global optimum since the objective is not convex (due to $\min_j$, as the minimum of multiple convex functions is not necessarily convex).

12.1.5 Importance of initialization

The standard k-means algorithm, as well as the variant that uses gradient descent, both are only guaranteed to converge to a local minimum, not necessarily a global minimum of the loss. Thus the answer we get out depends on how we initialize the cluster means.

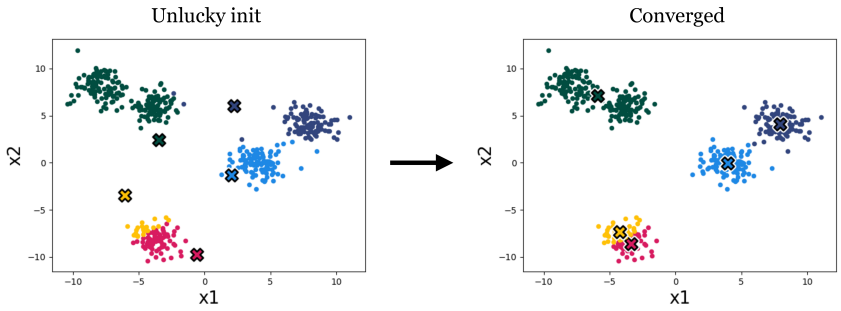


Figure 12.4: With the initialization of the means to the left, the yellow and red means end up splitting what perhaps should be one cluster in half.

Figure 12.4 is an example of a different initialization on our toy data, which results in a worse converged clustering:

A variety of methods have been developed to pick good initializations (for example, check out the *k-means++* algorithm). One simple option is to run the standard k-means algorithm multiple times, with different random initial conditions, and then pick from these the clustering that achieves the lowest k-means loss.

12.1.6 Importance of k

A very important choice in cluster algorithms is the number of clusters we are looking for. Some advanced algorithms can automatically infer a suitable number of clusters, but most of the time, like with k-means, we will have to pick k – it's a hyperparameter of the algorithm.

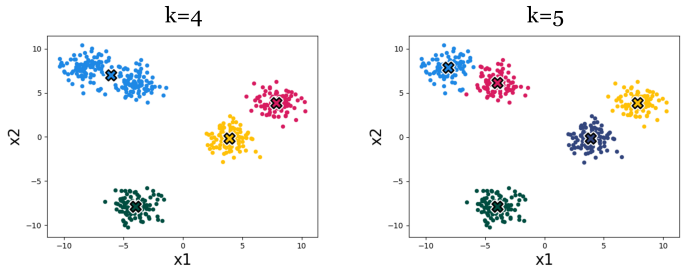


Figure 12.5: Example of k-means run on our toy data, with two different values of k. Setting k=4, on the left, results in one cluster being merged, compared to setting k=5, on the right. Which clustering do you think is better? How could you decide?

Figure 12.5 shows an example of the effect. Which result looks more correct? It can be hard to say! Using higher k we get more clusters, and with more clusters we can achieve lower within-cluster variance – the k-means objective will never increase, and will typically strictly decrease as we increase k. Eventually, we can increase k to equal the total number of datapoints, so that each datapoint is assigned to its own cluster. Then the k-means objective is zero, but the clustering reveals nothing. Clearly, then, we cannot use the k-means objective itself to choose the best value for k. In subsection 12.1.7.1, we will discuss some ways of evaluating the success of clustering beyond its ability to minimize the k-means objective, and it's with these sorts of methods that we might decide on a proper value of k.

Alternatively, you may be wondering: why bother picking a single k? Wouldn't it be nice to reveal a *hierarchy* of clusterings of our data, showing both coarse and fine groupings? Indeed *hierarchical clustering* is another important class of clustering algorithms, beyond k-means. These methods can be useful for discovering tree-like structure in data, and they work a bit like this: initially a coarse split/clustering of the data is applied at the root of the tree, and then as we descend the tree we split and cluster the data in ever more fine-grained ways. A prototypical example of hierarchical clustering is to discover a taxonomy of life, where creatures may be grouped at multiple granularities, from species to families to kingdoms.

12.1.7 k-means in feature space

Clustering algorithms group data based on a notion of *similarity*, and thus we need to define a *distance metric* between datapoints. This notion will also be useful in other machine learning approaches, such as nearest-neighbor methods that we see in Chapter 9. In k-means and other methods, our choice of distance metric can have a big impact on the results we will find.

Our k-means algorithm uses the Euclidean distance, i.e., $\left\|x^{(i)} - \mu^{(j)}\right\|$, with a loss function that is the square of this distance. We can modify k-means to use different distance metrics, but a more common trick is to stick with Euclidean distance but measured in a *feature space*. Just like we did for regression and classification problems, we can define a feature map from the data to a nicer feature representation, $\phi(x)$, and then apply k-means to cluster the data in the feature space.

In fact, using a simple distance metric in feature space can be equivalent to using a more sophisticated distance metric in the data space, and this trick forms the basis of *kernel methods*, which you can learn about in more advanced machine learning classes.

As a simple example, suppose we have two-dimensional data that is very stretched out in the first dimension and has less dynamic range in the second dimension. Then we may want to scale the dimensions so that each has similar dynamic range, prior to clustering. We could use standardization, like we did in Chapter 5.

If we want to cluster more complex data, like images, music, chemical compounds, etc., then we will usually need more sophisticated feature representations. One common practice these days is to use feature representations learned with a neural network. For example, we can use an autoencoder to compress images into feature vectors, then cluster those feature vectors.

12.1.7.1 How to evaluate clustering algorithms

One of the hardest aspects of clustering is knowing how to evaluate it. This is actually a big issue for all unsupervised learning methods, since we are just looking for patterns in the data, rather than explicitly trying to predict target values (which was the case with supervised learning).

Remember, evaluation metrics are *not* the same as loss functions, so we can't just measure success by looking at the k-means loss. In prediction problems, it is critical that the evaluation is on a held-out test set, while the loss is computed over training data. If we evaluate on training data we cannot detect overfitting. Something similar is going on with the example in Section 12.1.6, where setting k to be too large can precisely "fit" the data (minimize the loss), but yields no general insight.

One way to evaluate our clusters is to look at the **consistency** with which they are found when we run on different subsamples of our training data, or with different hyperparameters of our clustering algorithm (e.g., initializations). For example, if running on several bootstrapped samples (random subsets of our data) results in very different clusters, it should call into question the validity of any of the individual results.

If we have some notion of what **ground truth** clusters should be, e.g., a few data points that we know should be in the same cluster, then we can measure whether or not our

discovered clusters group these examples correctly.

Clustering is often used for **visualization** and **interpretability**, to make it easier for humans to understand the data. Here, human judgment may guide the choice of clustering algorithm. More quantitatively, discovered clusters may be used as input to **downstream tasks**. For example, we may fit a different regression function on the data within each cluster. Figure 12.6 gives an example where this might be useful. In cases like this, the success of a clustering algorithm can be indirectly measured based on the success of the downstream application (e.g., does it make the downstream predictions more accurate).

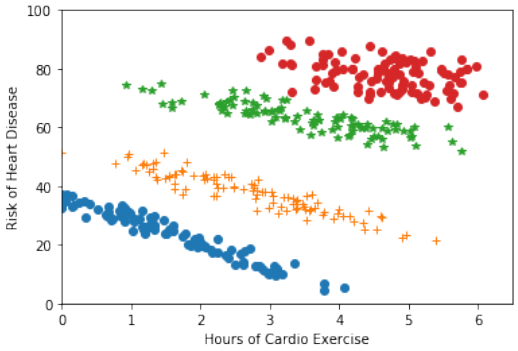


Figure 12.6: Averaged across the whole population, risk of heart disease positively correlates with hours of exercise. However, if we cluster the data, we can observe that there are four subgroups of the population which correspond to different age groups, and within each subgroup the correlation is negative. We can make better predictions, and better capture the presumed true effect, if we cluster this data and then model the trend in each cluster separately.

12.2 Autoencoder structure

Assume that we have input data $\mathcal{D} = \{x^{(1)}, \ldots, x^{(n)}\}$, where $x^{(i)} \in \mathbb{R}^d$. We seek to learn an autoencoder that will output a new dataset $\mathcal{D}_{out} = \{a^{(1)}, \ldots, a^{(n)}\}$, where $a^{(i)} \in \mathbb{R}^k$ with $k < d$. We can think about $a^{(i)}$ as the new *representation* of data point $x^{(i)}$. For example, in Fig. 12.7 we show the learned representations of a dataset of MNIST digits with $k = 2$. We see, after inspecting the individual data points, that unsupervised learning has found a compressed (or *latent*) representation where images of the same digit are close to each other, potentially greatly aiding subsequent clustering or classification tasks.

Formally, an autoencoder consists of two functions, a vector-valued *encoder* $g : \mathbb{R}^d \to \mathbb{R}^k$ that deterministically maps the data to the representation space $a \in \mathbb{R}^k$, and a *decoder* $h : \mathbb{R}^k \to \mathbb{R}^d$ that maps the representation space back into the original data space.

In general, the encoder and decoder functions might be any functions appropriate to the domain. Here, we are particularly interested in neural network embodiments of encoders and decoders. The basic architecture of one such autoencoder, consisting of only a single layer neural network in each of the encoder and decoder, is shown in Figure 12.8; note that bias terms W_0^1 and W_0^2 into the summation nodes exist, but are omitted for clarity in the figure. In this example, the original d-dimensional input is compressed into $k = 3$ dimensions via the encoder $g(x; W^1, W_0^1) = f_1({W^1}^T x + W_0^1)$ with $W^1 \in \mathbb{R}^{d \times k}$ and $W_0^1 \in \mathbb{R}^k$, and where the non-linearity f_1 is applied to each dimension of the vector. To recover (an approximation to) the original instance, we then apply the decoder $h(a; W^2, W_0^2) = f_2({W^2}^T a + W_0^2)$, where f_2 denotes a different non-linearity (activation function). In general, both the de-

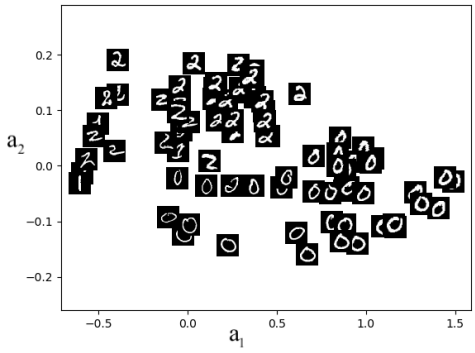


Figure 12.7: Compression of digits dataset into two dimensions. The input $x^{(i)}$, an image of a handwritten digit, is shown at the new low-dimensional representation (a_1, a_2).

coder and the encoder could involve multiple layers, as opposed to the single layer shown here. Learning seeks parameters W^1, W_0^1 and W^2, W_0^2 such that the reconstructed instances, $h(g(x^{(i)}; W^1, W_0^1); W^2, W_0^2)$, are close to the original input $x^{(i)}$.

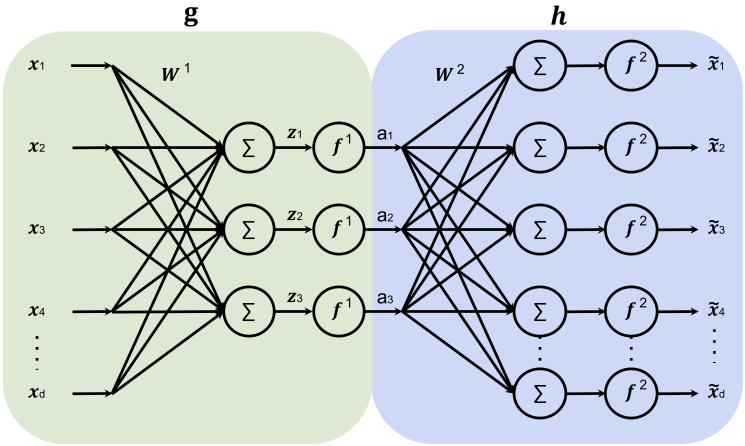


Figure 12.8: Autoencoder structure, showing the encoder (left half, light green), and the decoder (right half, light blue), encoding inputs x to the representation a, and decoding the representation to produce $\tilde{x}$, the reconstruction. In this specific example, the representation (a_1, a_2, a_3) only has three dimensions.

12.2.1 Autoencoder Learning

We learn the weights in an autoencoder using the same tools that we previously used for supervised learning, namely (stochastic) gradient descent of a multi-layer neural network to minimize a loss function. All that remains is to specify the loss function $\mathcal{L}(\tilde{x}, x)$, which tells us how to measure the discrepancy between the reconstruction $\tilde{x} = h(g(x; W^1, W_0^1); W^2, W_0^2)$ and the original input x. For example, for continuous-valued x it might make sense to use squared loss, i.e., $\mathcal{L}_{SE}(\tilde{x}, x) = \sum_{j=1}^{d} (x_j - \tilde{x}_j)^2$. Learning then seeks to optimize the parameters of h and g so as to minimize the reconstruction error, measured according to this loss function:

$$\min_{W^1, W_0^1, W^2, W_0^2} \sum_{i=1}^{n} \mathcal{L}_{SE}\left(h(g(x^{(i)}; W^1, W_0^1); W^2, W_0^2), x^{(i)}\right)$$

Alternatively, you could think of this as *multi-task learning*, where the goal is to predict each dimension of x. One can mix-and-match loss functions as appropriate for each dimension's data type.

12.2.2 Evaluating an autoencoder

What makes a good learned representation in an autoencoder? Notice that, without further constraints, it is always possible to perfectly reconstruct the input. For example, we could let $k = d$ and h and g be the identity functions. In this case, we would not obtain any compression of the data.

To learn something useful, we must create a *bottleneck* by making k to be smaller (often much smaller) than d. This forces the learning algorithm to seek transformations that describe the original data using as simple a description as possible. Thinking back to the digits dataset, for example, an example of a compressed representation might be the digit label (i.e., 0–9), rotation, and stroke thickness. Of course, there is no guarantee that the learning algorithm will discover precisely this representation. After learning, we can inspect the learned representations, such as by artificially increasing or decreasing one of the dimensions (e.g., a_1) and seeing how it affects the output $h(a)$, to try to better understand what it has learned.

As with clustering, autoencoders can be a preliminary step toward building other models, such as a regressor or classifier. For example, once a good encoder has been learned, the decoder might be replaced with another neural network that is then trained with supervised learning (perhaps using a smaller dataset that does include labels).

12.2.3 Linear encoders and decoders

We close by mentioning that even linear encoders and decoders can be very powerful. In this case, rather than minimizing the above objective with gradient descent, a technique called *principal components analysis* (PCA) can be used to obtain a closed-form solution to the optimization problem using a singular value decomposition (SVD). Just as a multilayer neural network with nonlinear activations for regression (learned by gradient descent) can be thought of as a nonlinear generalization of a linear regressor (fit by matrix algebraic operations), the neural network based autoencoders discussed above (and learned with gradient descent) can be thought of as a generalization of linear PCA (as solved with matrix algebra by SVD).

12.2.4 Advanced encoders and decoders

Advanced neural networks that build on the encoder-decoder conceptual decomposition have become increasingly powerful in recent years. One family of applications are *generative* networks, where new outputs that are "similar to" but different from any existing training sample are desired. In *variational autoencoders* the compressed representation encompasses information about the probability distribution of training samples, e.g., learning both mean and standard deviation variables in the bottleneck layer or latent representation. Then, new outputs can be generated by random sampling based on the latent representation variables and feeding those samples into the decoder. For instance, *Transformers* use *multiple* encoder and decoder blocks, together with a self-attention mechanism to make predictions about potential next outputs resulting from sequences of inputs. Such transformer networks have many applications in natural language processing and elsewhere.

APPENDIX A

Matrix derivative common cases

What are some conventions for derivatives of matrices and vectors? It will always work to explicitly write all indices and treat everything as scalars, but we introduce here some shortcuts that are often faster to use and helpful for understanding.

There are at least two consistent but different systems for describing shapes and rules for doing matrix derivatives. In the end, they all are correct, but it is important to be consistent.

We will use what is often called the 'Hessian' or denominator layout, in which we say that for

$\mathbf{x}$ of size $n \times 1$ and $\mathbf{y}$ of size $m \times 1$, $\partial \mathbf{y}/\partial \mathbf{x}$ is a matrix of size $n \times m$ with the (i, j) entry $\partial y_j/\partial x_i$. This denominator layout convention has been adopted by the field of machine learning to ensure that the shape of the gradient is the same as the shape of the shape of the respective derivative. This is somewhat controversial at large, but alas, we shall continue with denominator layout.

The discussion below closely follows the Wikipedia on matrix derivatives.

A.1 The shapes of things

Here are important special cases of the rule above:

- Scalar-by-scalar: For x of size 1×1 and y of size 1×1, $\partial y/\partial x$ is the (scalar) partial derivative of y with respect to x.
- Scalar-by-vector: For $\mathbf{x}$ of size $n \times 1$ and y of size 1×1, $\partial y/\partial \mathbf{x}$ (also written $\nabla_x y$, the gradient of y with respect to $\mathbf{x}$) is a column vector of size $n \times 1$ with the i^{th} entry $\partial y/\partial x_i$:

$$\partial y/\partial \mathbf{x} = \begin{bmatrix} \partial y/\partial x_1 \\ \partial y/\partial x_2 \\ \vdots \\ \partial y/\partial x_n \end{bmatrix}.$$

- Vector-by-scalar: For x of size 1×1 and $\mathbf{y}$ of size $m \times 1$, $\partial \mathbf{y}/\partial x$ is a row vector of size $1 \times m$ with the j^{th} entry $\partial \mathbf{y}_j/\partial x$:

$$\partial \mathbf{y}/\partial x = \begin{bmatrix} \partial y_1/\partial x & \partial y_2/\partial x & \cdots & \partial y_m/\partial x \end{bmatrix}.$$

- Vector-by-vector: For $\mathbf{x}$ of size $n \times 1$ and $\mathbf{y}$ of size $m \times 1$, $\partial \mathbf{y}/\partial \mathbf{x}$ is a matrix of size $n \times m$ with the (i, j) entry $\partial y_j/\partial x_i$:

$$\partial \mathbf{y}/\partial \mathbf{x} = \begin{bmatrix} \partial y_1/\partial x_1 & \partial y_2/\partial x_1 & \cdots & \partial y_m/\partial x_1 \\ \partial y_1/\partial x_2 & \partial y_2/\partial x_2 & \cdots & \partial y_m/\partial x_2 \\ \vdots & \vdots & \ddots & \vdots \\ \partial y_1/\partial x_n & \partial y_2/\partial x_n & \cdots & \partial y_m/\partial x_n \end{bmatrix}.$$

- Scalar-by-matrix: For $\mathbf{X}$ of size $n \times m$ and y of size 1×1, $\partial y/\partial \mathbf{X}$ (also written $\nabla_{\mathbf{X}} y$, the gradient of y with respect to $\mathbf{X}$) is a matrix of size $n \times m$ with the (i, j) entry $\partial y/\partial X_{i,j}$:

$$\partial y/\partial \mathbf{X} = \begin{bmatrix} \partial y/\partial X_{1,1} & \cdots & \partial y/\partial X_{1,m} \\ \vdots & \ddots & \vdots \\ \partial y/\partial X_{n,1} & \cdots & \partial y/\partial X_{n,m} \end{bmatrix}.$$

You may notice that in this list, we have not included matrix-by-matrix, matrix-by-vector, or vector-by-matrix derivatives. This is because, generally, they cannot be expressed nicely in matrix form and require higher order objects (e.g., tensors) to represent their derivatives. These cases are beyond the scope of this course.

Additionally, notice that for all cases, you can explicitly compute each element of the derivative object using (scalar) partial derivatives. You may find it useful to work through some of these by hand as you are reviewing matrix derivatives.

A.2 Some vector-by-vector identities

Here are some examples of $\partial \mathbf{y}/\partial \mathbf{x}$. In each case, assume $\mathbf{x}$ is $n \times 1$, $\mathbf{y}$ is $m \times 1$, a is a scalar constant, $\mathbf{a}$ is a vector that does not depend on $\mathbf{x}$ and $\mathbf{A}$ is a matrix that does not depend on $\mathbf{x}$, u and v are scalars that do depend on $\mathbf{x}$, and $\mathbf{u}$ and $\mathbf{v}$ are vectors that do depend on $\mathbf{x}$. We also have vector-valued functions $\mathbf{f}$ and $\mathbf{g}$.

A.2.1 Some fundamental cases

First, we will cover a couple of fundamental cases: suppose that $\mathbf{a}$ is an $m \times 1$ vector which is not a function of $\mathbf{x}$, an $n \times 1$ vector. Then,

$$\frac{\partial \mathbf{a}}{\partial \mathbf{x}} = \mathbf{0}, \tag{A.1}$$

is an $n \times m$ matrix of 0s. This is similar to the scalar case of differentiating a constant. Next, we can consider the case of differentiating a vector with respect to itself:

$$\frac{\partial \mathbf{x}}{\partial \mathbf{x}} = \mathbf{I} \tag{A.2}$$

This is the $n \times n$ identity matrix, with 1's along the diagonal and 0's elsewhere. It makes sense, because $\partial \mathbf{x}_j/\mathbf{x}_i$ is 1 for $i = j$ and 0 otherwise. This identity is also similar to the scalar case.

A.2.2 Derivatives involving a constant matrix

Let the dimensions of $\mathbf{A}$ be $m \times n$. Then the object $\mathbf{Ax}$ is an $m \times 1$ vector. We can then compute the derivative of $\mathbf{Ax}$ with respect to $\mathbf{x}$ as:

$$\frac{\partial \mathbf{Ax}}{\partial \mathbf{x}} = \begin{bmatrix} \partial(\mathbf{Ax})_1/\partial x_1 & \partial(\mathbf{Ax})_2/\partial x_1 & \cdots & \partial(\mathbf{Ax})_m/\partial x_1 \\ \partial(\mathbf{Ax})_1/\partial x_2 & \partial(\mathbf{Ax})_2/\partial x_2 & \cdots & \partial(\mathbf{Ax})_m/\partial x_2 \\ \vdots & \vdots & \ddots & \vdots \\ \partial(\mathbf{Ax})_1/\partial x_n & \partial(\mathbf{Ax})_2/\partial x_n & \cdots & \partial(\mathbf{Ax})_m/\partial x_n \end{bmatrix} \tag{A.3}$$

Note that any element of the column vector $\mathbf{Ax}$ can be written as, for $j = 1, \ldots, m$:

$$(\mathbf{Ax})_j = \sum_{k=1}^{n} A_{j,k} x_k.$$

Thus, computing the (i, j) entry of $\frac{\partial \mathbf{Ax}}{\partial \mathbf{x}}$ requires computing the partial derivative $\partial(\mathbf{Ax})_j/\partial x_i$:

$$\partial(\mathbf{Ax})_j/\partial x_i = \partial \left(\sum_{k=1}^{n} A_{j,k} x_k \right) /\partial x_i = A_{j,i}$$

Therefore, the (i, j) entry of $\frac{\partial \mathbf{Ax}}{\partial \mathbf{x}}$ is the (j, i) entry of $\mathbf{A}$:

$$\frac{\partial \mathbf{Ax}}{\partial \mathbf{x}} = \mathbf{A}^T \tag{A.4}$$

Similarly, for objects $\mathbf{x}, \mathbf{A}$ of the same shape, on can obtain,

$$\frac{\partial \mathbf{x}^T \mathbf{A}}{\partial \mathbf{x}} = \mathbf{A} \tag{A.5}$$

A.2.3 Linearity of derivatives

Suppose that $\mathbf{u}, \mathbf{v}$ are both vectors of size $m \times 1$. Then,

$$\frac{\partial(\mathbf{u} + \mathbf{v})}{\partial \mathbf{x}} = \frac{\partial \mathbf{u}}{\partial \mathbf{x}} + \frac{\partial \mathbf{v}}{\partial \mathbf{x}} \tag{A.6}$$

Suppose that a is a scalar constant and $\mathbf{u}$ is an $m \times 1$ vector that is a function of $\mathbf{x}$. Then,

$$\frac{\partial a\mathbf{u}}{\partial \mathbf{x}} = a\frac{\partial \mathbf{u}}{\partial \mathbf{x}} \tag{A.7}$$

One can extend the previous identity to vector- and matrix-valued constants. Suppose that $\mathbf{a}$ is a vector with shape $m \times 1$ and v is a scalar which depends on $\mathbf{x}$. Then,

$$\frac{\partial v\mathbf{a}}{\partial \mathbf{x}} = \frac{\partial v}{\partial \mathbf{x}}\mathbf{a}^T \tag{A.8}$$

First, checking dimensions, $\partial v/\partial \mathbf{x}$ is $n \times 1$ and $\mathbf{a}$ is $m \times 1$ so $\mathbf{a}^T$ is $1 \times m$ and our answer is $n \times m$ as it should be. Now, checking a value, element (i, j) of the answer is $\partial v\mathbf{a}_j/\partial x_i = (\partial v/\partial x_i)\mathbf{a}_j$ which corresponds to element (i, j) of $(\partial v/\partial \mathbf{x})\mathbf{a}^T$.

Similarly, suppose that $\mathbf{A}$ is a matrix which does not depend on $\mathbf{x}$ and $\mathbf{u}$ is a column vector which does depend on $\mathbf{x}$. Then,

$$\frac{\partial \mathbf{Au}}{\partial \mathbf{x}} = \frac{\partial \mathbf{u}}{\partial \mathbf{x}}\mathbf{A}^T \tag{A.9}$$

A.2.4 Product rule (vector-valued numerator)

Suppose that v is a scalar which depends on $\mathbf{x}$, while $\mathbf{u}$ is a column vector of shape $m \times 1$ and $\mathbf{x}$ is a column vector of shape $n \times 1$. Then,

$$\frac{\partial v\mathbf{u}}{\partial \mathbf{x}} = v\frac{\partial \mathbf{u}}{\partial \mathbf{x}} + \frac{\partial v}{\partial \mathbf{x}}\mathbf{u}^T \tag{A.10}$$

One can see this relationship by expanding the derivative as follows:

$$\frac{\partial v\mathbf{u}}{\partial \mathbf{x}} = \begin{bmatrix} \partial(vu_1)/\partial x_1 & \partial(vu_2)/\partial x_1 & \cdots & \partial(vu_m)/\partial x_1 \\ \partial(vu_1)/\partial x_2 & \partial(vu_2)/\partial x_2 & \cdots & \partial(vu_m)/\partial x_2 \\ \vdots & \vdots & \ddots & \vdots \\ \partial(vu_1)/\partial x_n & \partial(vu_2)/\partial x_n & \cdots & \partial(vu_m)/\partial x_n \end{bmatrix}.$$

Then, one can use the product rule for scalar-valued functions,

$$\partial(vu_j)/\partial x_i = v(\partial u_j/\partial x_i) + (\partial v/\partial x_i)u_j,$$

to obtain the desired result.

A.2.5 Chain rule

Suppose that $\mathbf{g}$ is a vector-valued function with output vector of shape $m \times 1$, and the argument to $\mathbf{g}$ is a column vector $\mathbf{u}$ of shape $d \times 1$ which depends on $\mathbf{x}$. Then, one can obtain the chain rule as,

$$\frac{\partial \mathbf{g}(\mathbf{u})}{\partial \mathbf{x}} = \frac{\partial \mathbf{u}}{\partial \mathbf{x}}\frac{\partial \mathbf{g}(\mathbf{u})}{\partial \mathbf{u}} \tag{A.11}$$

Following "the shapes of things," $\partial \mathbf{u}/\partial \mathbf{x}$ is $n \times d$ and $\partial \mathbf{g}(\mathbf{u})/\partial \mathbf{u}$ is $d \times m$, where element (i, j) is $\partial \mathbf{g}(\mathbf{u})_j/\partial \mathbf{u}_i$. The same chain rule applies for further compositions of functions:

$$\frac{\partial \mathbf{f}(\mathbf{g}(\mathbf{u}))}{\partial \mathbf{x}} = \frac{\partial \mathbf{u}}{\partial \mathbf{x}}\frac{\partial \mathbf{g}(\mathbf{u})}{\partial \mathbf{u}}\frac{\partial \mathbf{f}(\mathbf{g})}{\partial \mathbf{g}} \tag{A.12}$$

A.3 Some other identities

You can get many scalar-by-vector and vector-by-scalar cases as special cases of the rules above, making one of the relevant vectors just be 1 x 1. Here are some other ones that are handy. For more, see the Wikipedia article on Matrix derivatives (for consistency, only use the ones in *denominator layout*).

$$\frac{\partial \mathbf{u}^T\mathbf{v}}{\partial \mathbf{x}} = \frac{\partial \mathbf{u}}{\partial \mathbf{x}}\mathbf{v} + \frac{\partial \mathbf{v}}{\partial \mathbf{x}}\mathbf{u} \tag{A.13}$$

$$\frac{\partial \mathbf{u}^T}{\partial x} = \left(\frac{\partial \mathbf{u}}{\partial x}\right)^T \tag{A.14}$$

A.4 Derivation of gradient for linear regression

Applying identities A.5, A.13, A.6, A.4 A.1

$$\begin{aligned}\frac{\partial(\tilde{\mathbf{X}}\theta-\tilde{\mathbf{Y}})^T(\tilde{\mathbf{X}}\theta-\tilde{\mathbf{Y}})/n}{\partial\theta} &= \frac{2}{n}\frac{\partial(\tilde{\mathbf{X}}\theta-\tilde{\mathbf{Y}})}{\partial\theta}(\tilde{\mathbf{X}}\theta-\tilde{\mathbf{Y}}) \\ &= \frac{2}{n}\left(\frac{\partial\tilde{\mathbf{X}}\theta}{\partial\theta}-\frac{\partial\tilde{\mathbf{Y}}}{\partial\theta}\right)(\tilde{\mathbf{X}}\theta-\tilde{\mathbf{Y}}) \\ &= \frac{2}{n}\left(\tilde{\mathbf{X}}^T-\mathbf{0}\right)(\tilde{\mathbf{X}}\theta-\tilde{\mathbf{Y}}) \\ &= \frac{2}{n}\tilde{\mathbf{X}}^T(\tilde{\mathbf{X}}\theta-\tilde{\mathbf{Y}})\end{aligned}$$

A.5 Matrix derivatives using Einstein summation

You do not have to read or learn this! But you might find it interesting or helpful.

Consider the objective function for linear regression, written out as products of matrices:

$$J(\theta) = \frac{1}{n}(\tilde{X}\theta-\tilde{Y})^T(\tilde{X}\theta-\tilde{Y}), \tag{A.15}$$

where $\tilde{X} = X^T$ is $n \times d$, $\tilde{Y} = Y^T$ is $n \times 1$, and θ is $d \times 1$. How does one show, with no shortcuts, that

$$\nabla_\theta J = \frac{2}{n}\tilde{X}^T(\tilde{X}\theta-\tilde{Y}) \ ? \tag{A.16}$$

One neat way, which is very explicit, is to simply write all the matrices as variables with row and column indices, e.g., $\tilde{X}_{ab}$ is the row a, column b entry of the matrix $\tilde{X}$. Furthermore, let us use the convention that in any product, all indices which appear more than once get summed over; this is a popular convention in theoretical physics, and lets us suppress all the summation symbols which would otherwise clutter the following expressions. For example, $\tilde{X}_{ab}\theta_b$ would be the implicit summation notation giving the element at the a^{th} row of the matrix-vector product $\tilde{X}\theta$.

Using implicit summation notation with explicit indices, we can rewrite $J(\theta)$ as

$$J(\theta) = \frac{1}{n}\left(\tilde{X}_{ab}\theta_b-\tilde{Y}_a\right)\left(\tilde{X}_{ac}\theta_c-\tilde{Y}_a\right). \tag{A.17}$$

Note that we no longer need the transpose on the first term, because all that transpose accomplished was to take a dot product between the vector given by the left term, and the vector given by the right term. With implicit summation, this is accomplished by the two terms sharing the repeated index a.

Taking the derivative of J with respect to the d^{th} element of θ thus gives, using the chain rule for (ordinary scalar) multiplication:

$$\begin{aligned}\frac{dJ}{d\theta_d} &= \frac{1}{n}\left[\tilde{X}_{ab}\delta_{bd}\left(\tilde{X}_{ac}\theta_c-\tilde{Y}_a\right)+\left(\tilde{X}_{ab}\theta_b-\tilde{Y}_a\right)\tilde{X}_{ac}\delta_{cd}\right] &\text{(A.18)}\\ &= \frac{1}{n}\left[\tilde{X}_{ad}\left(\tilde{X}_{ac}\theta_c-\tilde{Y}_a\right)+\left(\tilde{X}_{ab}\theta_b-\tilde{Y}_a\right)\tilde{X}_{ad}\right] &\text{(A.19)}\\ &= \frac{2}{n}\tilde{X}_{ad}\left(\tilde{X}_{ab}\theta_b-\tilde{Y}_a\right), &\text{(A.20)}\end{aligned}$$

where the second line follows from the first, with the definition that $\delta_{bd} = 1$ only when $b = d$ (and similarly for δ_{cd}). And the third line follows from the second by recognizing that the two terms in the second line are identical. Now note that in this implicit summation notation, the a, b element of the matrix product of A and B is $(AB)_{ac} = A_{ab}B_{bc}$. That is, ordinary matrix multiplication sums over indices which are adjacent to each other, because a row of A times a column of B becomes a scalar number. So the term in the above equation with $\tilde{X}_{ad}\tilde{X}_{ab}$ is not a matrix product of $\tilde{X}$ with $\tilde{X}$. However, taking the transpose $\tilde{X}^T$ switches row and column indices, so $\tilde{X}_{ad} = \tilde{X}^T_{da}$. And $\tilde{X}^T_{da}\tilde{X}_{ab}$ *is* a matrix product of $\tilde{X}^T$ with $\tilde{X}$! Thus, we have that

$$\frac{dJ}{d\theta_d} = \frac{2}{n}\tilde{X}^T_{da}\left(\tilde{X}_{ab}\theta_b - \tilde{Y}_a\right) \tag{A.21}$$

$$= \frac{2}{n}\left[\tilde{X}^T\left(\tilde{X}\theta - \tilde{Y}\right)\right]_d , \tag{A.22}$$

which is the desired result.

APPENDIX B

Optimizing Neural Networks

B.0.1 Strategies towards adaptive step-size

B.0.1.1 Running averages

We'll start by looking at the notion of a *running average*. It's a computational strategy for estimating a possibly weighted average of a sequence of data. Let our data sequence be $c_1, c_2, \ldots$; then we define a sequence of running average values, $C_0, C_1, C_2, \ldots$ using the equations

$$\begin{aligned} C_0 &= 0 \\ C_t &= \gamma_t C_{t-1} + (1-\gamma_t)c_t \end{aligned}$$

where $\gamma_t \in (0,1)$. If γ_t is a constant, then this is a *moving* average, in which

$$\begin{aligned} C_T &= \gamma C_{T-1} + (1-\gamma)c_T \\ &= \gamma(\gamma C_{T-2} + (1-\gamma)c_{T-1}) + (1-\gamma)c_T \\ &= \sum_{t=1}^{T} \gamma^{T-t}(1-\gamma)c_t \end{aligned}$$

So, you can see that inputs c_t closer to the end of the sequence T have more effect on C_T than early inputs.

If, instead, we set $\gamma_t = (t-1)/t$, then we get the actual average.

Study Question: Prove to yourself that the previous assertion holds.

B.0.1.2 Momentum

Now, we can use methods that are a bit like running averages to describe strategies for computing η. The simplest method is *momentum*, in which we try to "average" recent gradient updates, so that if they have been bouncing back and forth in some direction, we take out that component of the motion. For momentum, we have

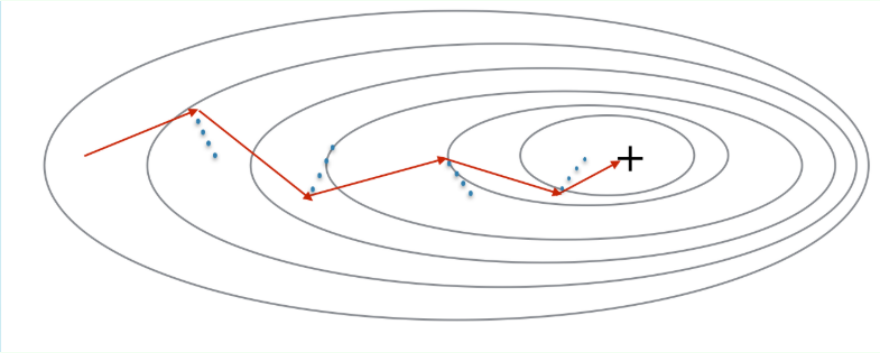
$$\begin{aligned} V_0 &= 0 \\ V_t &= \gamma V_{t-1} + \eta \nabla_W J(W_{t-1}) \\ W_t &= W_{t-1} - V_t \end{aligned}$$

This doesn't quite look like an adaptive step size. But what we can see is that, if we let $\eta = \eta'(1-\gamma)$, then the rule looks exactly like doing an update with step size η' on a moving average of the gradients with parameter γ:

$$\begin{aligned} M_0 &= 0 \\ M_t &= \gamma M_{t-1} + (1-\gamma)\nabla_W J(W_{t-1}) \\ W_t &= W_{t-1} - \eta' M_t \end{aligned}$$

Study Question: Prove to yourself that these formulations are equivalent.

We will find that V_t will be bigger in dimensions that consistently have the same sign for ∇_W and smaller for those that don't. Of course we now have *two* parameters to set (η and γ), but the hope is that the algorithm will perform better overall, so it will be worth trying to find good values for them. Often γ is set to be something like 0.9.



The red arrows show the update after each successive step of mini-batch gradient descent with momentum. The blue points show the direction of the gradient with respect to the mini-batch at each step. Momentum smooths the path taken towards the local minimum and leads to faster convergence.

Study Question: If you set $\gamma = 0.1$, would momentum have more of an effect or less of an effect than if you set it to 0.9?

B.0.1.3 Adadelta

Another useful idea is this: we would like to take larger steps in parts of the space where $J(W)$ is nearly flat (because there's no risk of taking too big a step due to the gradient being large) and smaller steps when it is steep. We'll apply this idea to each weight independently, and end up with a method called *adadelta*, which is a variant on *adagrad* (for adaptive gradient). Even though our weights are indexed by layer, input unit and output unit, for simplicity here, just let W_j be any weight in the network (we will do the same thing for all of them).

$$\begin{aligned} g_{t,j} &= \nabla_W J(W_{t-1})_j \\ G_{t,j} &= \gamma G_{t-1,j} + (1-\gamma) g_{t,j}^2 \\ W_{t,j} &= W_{t-1,j} - \frac{\eta}{\sqrt{G_{t,j} + \epsilon}} g_{t,j} \end{aligned}$$

The sequence $G_{t,j}$ is a moving average of the square of the jth component of the gradient. We square it in order to be insensitive to the sign—we want to know whether the magnitude is big or small. Then, we perform a gradient update to weight j, but divide the step size by $\sqrt{G_{t,j} + \epsilon}$, which is larger when the surface is steeper in direction j at point W_{t-1} in weight space; this means that the step size will be smaller when it's steep and larger when it's flat.

B.0.1.4 Adam

Adam has become the default method of managing step sizes in neural networks. It combines the ideas of momentum and adadelta. We start by writing moving averages of the gradient and squared gradient, which reflect estimates of the mean and variance of the gradient for weight j:

Although, interestingly, it may actually violate the convergence conditions of SGD: arxiv.org/abs/1705.08292

$$\begin{aligned} g_{t,j} &= \nabla_W J(W_{t-1})_j \\ m_{t,j} &= B_1 m_{t-1,j} + (1 - B_1) g_{t,j} \\ v_{t,j} &= B_2 v_{t-1,j} + (1 - B_2) g_{t,j}^2 \quad . \end{aligned}$$

A problem with these estimates is that, if we initialize $m_0 = v_0 = 0$, they will always be biased (slightly too small). So we will correct for that bias by defining

$$\begin{aligned} \hat{m}_{t,j} &= \frac{m_{t,j}}{1 - B_1^t} \\ \hat{v}_{t,j} &= \frac{v_{t,j}}{1 - B_2^t} \\ W_{t,j} &= W_{t-1,j} - \frac{\eta}{\sqrt{\hat{v}_{t,j} + \epsilon}} \hat{m}_{t,j} \quad . \end{aligned}$$

Note that B_1^t is B_1 raised to the power t, and likewise for B_2^t. To justify these corrections, note that if we were to expand $m_{t,j}$ in terms of $m_{0,j}$ and $g_{0,j}, g_{1,j}, \ldots, g_{t,j}$ the coefficients would sum to 1. However, the coefficient behind $m_{0,j}$ is B_1^t and since $m_{0,j} = 0$, the sum of coefficients of non-zero terms is $1 - B_1^t$, hence the correction. The same justification holds for $v_{t,j}$.

Now, our update for weight j has a step size that takes the steepness into account, as in adadelta, but also tends to move in the same direction, as in momentum. The authors of this method propose setting $B_1 = 0.9, B_2 = 0.999, \epsilon = 10^{-8}$. Although we now have even more parameters, Adam is not highly sensitive to their values (small changes do not have a huge effect on the result).

Study Question: Define $\hat{m}_j$ directly as a moving average of $g_{t,j}$. What is the decay (γ parameter)?

Even though we now have a step-size for each weight, and we have to update various quantities on each iteration of gradient descent, it's relatively easy to implement by maintaining a matrix for each quantity ($m_t^\ell, v_t^\ell, g_t^\ell, {g_t^2}^\ell$) in each layer of the network.

B.0.2 Batch Normalization Details

Let's think of the batch-norm layer as taking Z^l as input and producing an output $\widehat{Z}^l$ as output. But now, instead of thinking of Z^l as an $n^l \times 1$ vector, we have to explicitly think about handling a mini-batch of data of size K, all at once, so Z^l will be $n^l \times K$, and so will the output $\widehat{Z}^l$.

Our first step will be to compute the *batchwise* mean and standard deviation. Let μ^l be the $n^l \times 1$ vector where

$$\mu_i^l = \frac{1}{K} \sum_{j=1}^{K} Z_{ij}^l \ ,$$

and let σ^l be the $n^l \times 1$ vector where

$$\sigma_i^l = \sqrt{\frac{1}{K} \sum_{j=1}^{K} (Z_{ij}^l - \mu_i)^2} \ .$$

The basic normalized version of our data would be a matrix, element (i, j) of which is

$$\overline{Z}_{ij}^l = \frac{Z_{ij}^l - \mu_i^l}{\sigma_i^l + \epsilon} \ ,$$

where ϵ is a very small constant to guard against division by zero. However, if we let these be our $\widehat{Z}^l$ values, we really are forcing something too strong on our data—our goal was to normalize across the data batch, but not necessarily force the output values to have exactly mean 0 and standard deviation 1. So, we will give the layer the "opportunity" to shift and scale the outputs by adding new weights to the layer. These weights are G^l and B^l, each of which is an $n^l \times 1$ vector. Using the weights, we define the final output to be

$$\widehat{Z}_{ij}^l = G_i^l \overline{Z}_{ij}^l + B_i^l \ .$$

That's the forward pass. Whew!

Now, for the backward pass, we have to do two things: given $\partial L / \partial \widehat{Z}^l$,

- Compute $\partial L / \partial Z^l$ for back-propagation, and
- Compute $\partial L / \partial G^l$ and $\partial L / \partial B^l$ for gradient updates of the weights in this layer.

Schematically

$$\frac{\partial L}{\partial B} = \frac{\partial L}{\partial \widehat{Z}} \frac{\partial \widehat{Z}}{\partial B} \ .$$

For simplicity we will drop the reference to the layer l in the rest of the derivation

It's hard to think about these derivatives in matrix terms, so we'll see how it works for the components. B_i contributes to $\widehat{Z}_{ij}$ for all data points j in the batch. So

$$\begin{aligned} \frac{\partial L}{\partial B_i} &= \sum_j \frac{\partial L}{\partial \widehat{Z}_{ij}} \frac{\partial \widehat{Z}_{ij}}{\partial B_i} \\ &= \sum_j \frac{\partial L}{\partial \widehat{Z}_{ij}} \ , \end{aligned}$$

Similarly, G_i contributes to $\widehat{Z}_{ij}$ for all data points j in the batch. So

$$\begin{aligned} \frac{\partial L}{\partial G_i} &= \sum_j \frac{\partial L}{\partial \widehat{Z}_{ij}} \frac{\partial \widehat{Z}_{ij}}{\partial G_i} \\ &= \sum_j \frac{\partial L}{\partial \widehat{Z}_{ij}} \overline{Z}_{ij} \ . \end{aligned}$$

Now, let's figure out how to do backprop. We can start schematically:

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial \widehat{Z}} \frac{\partial \widehat{Z}}{\partial Z} \ .$$

And because dependencies only exist across the batch, but not across the unit outputs,

$$\frac{\partial L}{\partial Z_{ij}} = \sum_{k=1}^{K} \frac{\partial L}{\partial \widehat{Z}_{ik}} \frac{\partial \widehat{Z}_{ik}}{\partial Z_{ij}} \quad .$$

The next step is to note that

$$\begin{aligned} \frac{\partial \widehat{Z}_{ik}}{\partial Z_{ij}} &= \frac{\partial \widehat{Z}_{ik}}{\partial \overline{Z}_{ik}} \frac{\partial \overline{Z}_{ik}}{\partial Z_{ij}} \\ &= G_i \frac{\partial \overline{Z}_{ik}}{\partial Z_{ij}} \quad . \end{aligned}$$

And now that

$$\frac{\partial \overline{Z}_{ik}}{\partial Z_{ij}} = \left(\delta_{jk} - \frac{\partial \mu_i}{\partial Z_{ij}} \right) \frac{1}{\sigma_i} - \frac{Z_{ik} - \mu_i}{\sigma_i^2} \frac{\partial \sigma_i}{\partial Z_{ij}} \quad ,$$

where $\delta_{jk} = 1$ if $j = k$ and $\delta_{jk} = 0$ otherwise. Getting close! We need two more small parts:

$$\begin{aligned} \frac{\partial \mu_i}{\partial Z_{ij}} &= \frac{1}{K} \, , \\ \frac{\partial \sigma_i}{\partial Z_{ij}} &= \frac{Z_{ij} - \mu_i}{K \sigma_i} \quad . \end{aligned}$$

Putting the whole crazy thing together, we get

$$\frac{\partial L}{\partial Z_{ij}} = \sum_{k=1}^{K} \frac{\partial L}{\partial \widehat{Z}_{ik}} G_i \frac{1}{K \sigma_i} \left(\delta_{jk} K - 1 - \frac{(Z_{ik} - \mu_i)(Z_{ij} - \mu_i)}{\sigma_i^2} \right) \quad .$$

APPENDIX C

Recurrent Neural Networks

So far, we have limited our attention to domains in which each output y is assumed to have been generated as a function of an associated input x, and our hypotheses have been "pure" functions, in which the output depends only on the input (and the parameters we have learned that govern the function's behavior). In the next few chapters, we are going to consider cases in which our models need to go beyond functions. In particular, behavior as a function of *time* will be an important concept:

- In *recurrent neural networks*, the hypothesis that we learn is not a function of a single input, but of the whole sequence of inputs that the predictor has received.
- In *reinforcement learning*, the hypothesis is either a *model* of a domain (such as a game) as a recurrent system or a *policy* which is a pure function, but whose loss is determined by the ways in which the policy interacts with the domain over time.

In this chapter, we introduce *state machines*. We start with deterministic state machines, and then consider recurrent neural network (RNN) architectures to model their behavior. Later, in Chapter 10, we will study *Markov decision processes* (MDPs) that extend to consider probabilistic (rather than deterministic) transitions in our state machines. RNNs and MDPs will enable description and modeling of temporally sequential patterns of behavior that are important in many domains.

C.1 State machines

A *state machine* is a description of a process (computational, physical, economic) in terms of its potential sequences of *states*.

This is such a pervasive idea that it has been given many names in many subareas of computer science, control theory, physics, etc., including: *automaton*, *transducer*, *dynamical system*, etc.

The *state* of a system is defined to be all you would need to know about the system to predict its future trajectories as well as possible. It could be the position and velocity of an object or the locations of your pieces on a game board, or the current traffic densities on a highway network.

Formally, we define a *state machine* as $(\mathcal{S}, \mathcal{X}, \mathcal{Y}, s_0, f_s, f_o)$ where

- $\mathcal{S}$ is a finite or infinite set of possible states;
- $\mathcal{X}$ is a finite or infinite set of possible inputs;

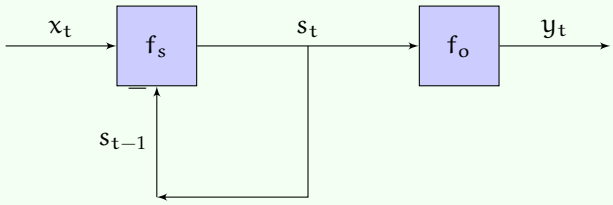
- $\mathcal{Y}$ is a finite or infinite set of possible outputs;
- $s_0 \in \mathcal{S}$ is the initial state of the machine;
- $f_s : \mathcal{S} \times \mathcal{X} \to \mathcal{S}$ is a *transition function*, which takes an input and a previous state and produces a next state;
- $f_o : \mathcal{S} \to \mathcal{Y}$ is an *output function*, which takes a state and produces an output.

The basic operation of the state machine is to start with state s_0, then iteratively compute for $t \geqslant 1$:

In some cases, we will pick a starting state from a set or distribution.

$$s_t = f_s(s_{t-1}, x_t) \tag{C.1}$$
$$y_t = f_o(s_t) \tag{C.2}$$

The diagram below illustrates this process. Note that the "feedback" connection of s_t back into f_s has to be buffered or delayed by one time step—-otherwise what it is computing would not generally be well defined.



So, given a sequence of inputs $x_1, x_2, \ldots$ the machine generates a sequence of outputs

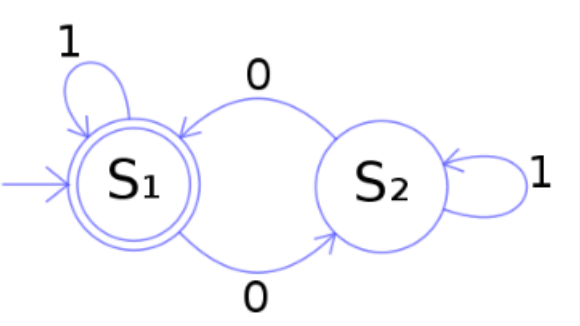
$$\underbrace{f_o(f_s(s_0, x_1))}_{y_1}, \underbrace{f_o(f_s(f_s(s_0, x_1), x_2))}_{y_2}, \ldots \quad .$$

We sometimes say that the machine *transduces* sequence x into sequence y. The output at time t can have dependence on inputs from steps 1 to t.

There are a huge number of major and minor variations on the idea of a state machine. We'll just work with one specific one in this section and another one in the next, but don't worry if you see other variations out in the world!

One common form is *finite state machines*, in which $\mathcal{S}$, $\mathcal{X}$, and $\mathcal{Y}$ are all finite sets. They are often described using *state transition diagrams* such as the one below, in which nodes stand for states and arcs indicate transitions. Nodes are labeled by which output they generate and arcs are labeled by which input causes the transition.

One can verify that the state machine below reads binary strings and determines the parity of the number of zeros in the given string. Check for yourself that all input binary strings end in state S_1 if and only if they contain an even number of zeros.



All computers can be described, at the digital level, as finite state machines. Big, but finite!

Another common structure that is simple but powerful and used in signal processing and control is *linear time-invariant (LTI) systems*. In this case, all the quantities are real-valued vectors: $\mathcal{S} = \mathbb{R}^m$, $\mathcal{X} = \mathbb{R}^l$ and $\mathcal{Y} = \mathbb{R}^n$. The functions f_s and f_o are linear functions of their inputs. The transition function is described by the state matrix A and the input matrix B; the output function is defined by the output matrix C, each with compatible dimensions. In discrete time, they can be defined by a linear difference equation, like

$$s_t = f_s(s_{t-1}, x_t) = As_{t-1} + Bx_t, \tag{C.3}$$
$$y_t = f_o(s_t) = Cs_t, \tag{C.4}$$

and can be implemented using state to store relevant previous input and output information. We will study *recurrent neural networks* which are a lot like a non-linear version of an LTI system.

C.2 Recurrent neural networks

In Chapter 6, we studied neural networks and how the weights of a network can be obtained by training on data, so that the neural network will model a function that approximates the relationship between the (x, y) pairs in a supervised-learning training set. In Section C.1 above, we introduced state machines to describe sequential temporal behavior. Here in Section C.2, we explore recurrent neural networks by defining the architecture and weight matrices in a neural network to enable modeling of such state machines. Then, in Section C.3, we present a loss function that may be employed for training *sequence to sequence* RNNs, and then consider application to language translation and recognition in Section C.4. In Section C.5, we'll see how to use gradient-descent methods to train the weights of an RNN so that it performs a *transduction* that matches as closely as possible a training set of input-output *sequences*.

A *recurrent neural network* is a state machine with neural networks constituting functions f_s and f_o:

$$s_t = f_s\left(W^{sx}x_t + W^{ss}s_{t-1} + W_0^{ss}\right) \tag{C.5}$$
$$y_t = f_o\left(W^o s_t + W_0^o\right) \quad . \tag{C.6}$$

The inputs, states, and outputs are all vector-valued:

$$x_t : \ell \times 1 \tag{C.7}$$
$$s_t : m \times 1 \tag{C.8}$$
$$y_t : n \times 1 \quad . \tag{C.9}$$

The weights in the network, then, are

$$W^{sx} : m \times \ell \tag{C.10}$$
$$W^{ss} : m \times m \tag{C.11}$$
$$W_0^{ss} : m \times 1 \tag{C.12}$$
$$W^o : n \times m \tag{C.13}$$
$$W_0^o : n \times 1 \tag{C.14}$$

with activation functions f_s and f_o.

Study Question: Check dimensions here to be sure it all works out. Remember that we apply f_s and f_o elementwise, unless f_o is a softmax activation.

C.3 Sequence-to-sequence RNN

Now, how can we set up an RNN to model and be trained to produce a transduction of one sequence to another? This problem is sometimes called *sequence-to-sequence* mapping. You can think of it as a kind of regression problem: given an input sequence, learn to generate the corresponding output sequence.

A training set has the form $\left[\left(x^{(1)}, y^{(1)}\right), \ldots, \left(x^{(q)}, y^{(q)}\right)\right]$, where

- $x^{(i)}$ and $y^{(i)}$ are length $n^{(i)}$ sequences;
- sequences in the *same pair* are the same length; and sequences in different pairs may have different lengths.

One way to think of training a sequence **classifier** is to reduce it to a transduction problem, where $y_t = 1$ if the sequence $x_1, \ldots, x_t$ is a *positive* example of the class of sequences and -1 otherwise.

Next, we need a loss function. We start by defining a loss function on sequences. There are many possible choices, but usually it makes sense just to sum up a per-element loss function on each of the output values, where g is the predicted sequence and y is the actual one:

$$\mathcal{L}_{\text{seq}}\left(g^{(i)}, y^{(i)}\right) = \sum_{t=1}^{n^{(i)}} \mathcal{L}_{\text{elt}}\left(g_t^{(i)}, y_t^{(i)}\right) \quad . \tag{C.15}$$

The per-element loss function $\mathcal{L}_{\text{elt}}$ will depend on the type of y_t and what information it is encoding, in the same way as for a supervised network.

So it could be NLL, squared loss, etc.

Then, letting $W = (W^{sx}, W^{ss}, W^{o}, W_0^{ss}, W_0^{o})$, our overall goal is to minimize the objective

$$J(W) = \frac{1}{q}\sum_{i=1}^{q} \mathcal{L}_{\text{seq}}\left(\text{RNN}(x^{(i)}; W), y^{(i)}\right) \quad , \tag{C.16}$$

where $\text{RNN}(x; W)$ is the output sequence generated, given input sequence x.

It is typical to choose f_s to be *tanh* but any non-linear activation function is usable. We choose f_o to align with the types of our outputs and the loss function, just as we would do in regular supervised learning.

Remember that it looks like a sigmoid but ranges from -1 to +1.

C.4 RNN as a language model

A *language model* is a sequence to sequence RNN which is trained on a token sequence of the form, $c = (c_1, c_2, \ldots, c_k)$, and is used to predict the next token $c_t, t \leqslant k$, given a sequence of the previous $(t-1)$ tokens:

A "token" is generally a character, common word fragment, or a word.

$$c_t = \text{RNN}\left((c_1, c_2, \ldots, c_{t-1}); W\right) \tag{C.17}$$

We can convert this to a sequence-to-sequence training problem by constructing a data set of q different (x, y) sequence pairs, where we make up new special tokens, start and end, to signal the beginning and end of the sequence:

$$x = (\langle\text{start}\rangle, c_1, c_2, \ldots, c_k) \tag{C.18}$$

$$y = (c_1, c_2, \ldots, \langle\text{end}\rangle) \tag{C.19}$$

C.5 Back-propagation through time

Now the fun begins! We can now try to find a W to minimize J using gradient descent. We will work through the simplest method, *back-propagation through time* (BPTT), in detail. This is generally not the best method to use, but it's relatively easy to understand. In Section C.6 we will sketch alternative methods that are in much more common use.

What we want you to take away from this section is that, by "unrolling" a recurrent network out to model a particular sequence, we can treat the whole thing as a feed-forward network with a lot of parameter sharing. Thus, we can tune the parameters using stochastic gradient descent, and learn to model sequential mappings. The concepts here are very important. While the details are important to get right if you need to implement something, we present the mathematical details below primarily to convey or explain the larger concepts.

Calculus reminder: total derivative Most of us are not very careful about the difference between the *partial derivative* and the *total derivative*. We are going to use a nice example from the Wikipedia article on partial derivatives to illustrate the difference. The volume of a circular cone depends on its height and radius:

$$V(r, h) = \frac{\pi r^2 h}{3} \quad . \tag{C.20}$$

The partial derivatives of volume with respect to height and radius are

$$\frac{\partial V}{\partial r} = \frac{2\pi r h}{3} \quad \text{and} \quad \frac{\partial V}{\partial h} = \frac{\pi r^2}{3} \quad . \tag{C.21}$$

They measure the change in V assuming everything is held constant except the single variable we are changing. Now assume that we want to preserve the cone's proportions in the sense that the ratio of radius to height stays constant. Then we can't really change one without changing the other. In this case, we really have to think about the *total derivative*. If we're interested in the total derivative with respect to r, we sum the "paths" along which r might influence V:

$$\frac{dV}{dr} = \frac{\partial V}{\partial r} + \frac{\partial V}{\partial h}\frac{dh}{dr} \tag{C.22}$$

$$= \frac{2\pi r h}{3} + \frac{\pi r^2}{3}\frac{dh}{dr} \tag{C.23}$$

Or if we're interested in the total derivative with respect to h, we consider how h might influence V, either directly or via r:

$$\frac{dV}{dh} = \frac{\partial V}{\partial h} + \frac{\partial V}{\partial r}\frac{dr}{dh} \tag{C.24}$$

$$= \frac{\pi r^2}{3} + \frac{2\pi r h}{3}\frac{dr}{dh} \tag{C.25}$$

Just to be completely concrete, let's think of a right circular cone with a fixed angle $\alpha = \tan r/h$, so that if we change r or h then α remains constant. So we have $r = h\tan^{-1}\alpha$; let constant $c = \tan^{-1}\alpha$, so now $r = ch$. Thus, we finally have

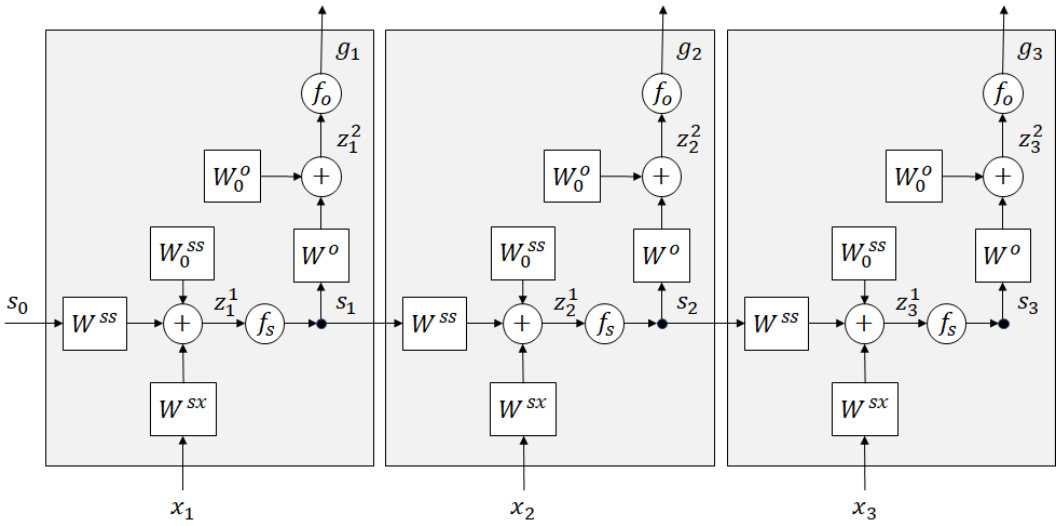
$$\frac{dV}{dr} = \frac{2\pi r h}{3} + \frac{\pi r^2}{3}\frac{1}{c} \tag{C.26}$$

$$\frac{dV}{dh} = \frac{\pi r^2}{3} + \frac{2\pi r h}{3}c \, . \tag{C.27}$$

The BPTT process goes like this:

(1) Sample a training pair of sequences (x, y); let their length be n.

(2) "Unroll" the RNN to be length n (picture for $n = 3$ below), and initialize s_0:



Now, we can see our problem as one of performing what is almost an ordinary back-propagation training procedure in a feed-forward neural network, but with the difference that the weight matrices are shared among the layers. In many ways, this is similar to what ends up happening in a convolutional network, except in the convnet, the weights are re-used spatially, and here, they are re-used temporally.

(3) Do the *forward pass*, to compute the predicted output sequence g:

$$z_t^1 = W^{sx} x_t + W^{ss} s_{t-1} + W_0^{ss} \tag{C.28}$$

$$s_t = f_s(z_t^1) \tag{C.29}$$

$$z_t^2 = W^o s_t + W_0^o \tag{C.30}$$

$$g_t = f_o(z_t^2) \tag{C.31}$$

(4) Do *backward pass* to compute the gradients. For both W^{ss} and W^{sx} we need to find

$$\frac{d\mathcal{L}_{seq}(g, y)}{dW} = \sum_{u=1}^{n} \frac{d\mathcal{L}_{elt}(g_u, y_u)}{dW}$$

Letting $\mathcal{L}_u = \mathcal{L}_{elt}(g_u, y_u)$ and using the *total derivative*, which is a sum over all the ways in which W affects $\mathcal{L}_u$, we have

$$= \sum_{u=1}^{n} \sum_{t=1}^{n} \frac{\partial s_t}{\partial W} \frac{\partial \mathcal{L}_u}{\partial s_t}$$

Re-organizing, we have

$$= \sum_{t=1}^{n} \frac{\partial s_t}{\partial W} \sum_{u=1}^{n} \frac{\partial \mathcal{L}_u}{\partial s_t}$$

Because s_t only affects $\mathcal{L}_t, \mathcal{L}_{t+1}, \ldots, \mathcal{L}_n$,

$$= \sum_{t=1}^{n} \frac{\partial s_t}{\partial W} \sum_{u=t}^{n} \frac{\partial \mathcal{L}_u}{\partial s_t}$$

$$= \sum_{t=1}^{n} \frac{\partial s_t}{\partial W} \left(\frac{\partial \mathcal{L}_t}{\partial s_t} + \underbrace{\sum_{u=t+1}^{n} \frac{\partial \mathcal{L}_u}{\partial s_t}}_{\delta^{s_t}} \right) . \tag{C.32}$$

where δ^{s_t} is the dependence of the future loss (incurred after step t) on the state S_t. We can compute this backwards, with t going from n down to 1. The trickiest part is figuring out how early states contribute to later losses. We define the *future loss* after step t to be

That is, δ^{s_t} is how much we can blame state s_t for all the future element losses.

$$F_t = \sum_{u=t+1}^{n} \mathcal{L}_{\text{elt}}(g_u, y_u) \ , \tag{C.33}$$

so

$$\delta^{s_t} = \frac{\partial F_t}{\partial s_t} \ . \tag{C.34}$$

At the last stage, $F_n = 0$ so $\delta^{s_n} = 0$.

Now, working backwards,

$$\delta^{s_{t-1}} = \frac{\partial}{\partial s_{t-1}} \sum_{u=t}^{n} \mathcal{L}_{\text{elt}}(g_u, y_u) \tag{C.35}$$

$$= \frac{\partial s_t}{\partial s_{t-1}} \frac{\partial}{\partial s_t} \sum_{u=t}^{n} \mathcal{L}_{\text{elt}}(g_u, y_u) \tag{C.36}$$

$$= \frac{\partial s_t}{\partial s_{t-1}} \frac{\partial}{\partial s_t} \left[\mathcal{L}_{\text{elt}}(g_t, y_t) + \sum_{u=t+1}^{n} \mathcal{L}_{\text{elt}}(g_u, y_u) \right] \tag{C.37}$$

$$= \frac{\partial s_t}{\partial s_{t-1}} \left[\frac{\partial \mathcal{L}_{\text{elt}}(g_t, y_t)}{\partial s_t} + \delta^{s_t} \right] \tag{C.38}$$

Now, we can use the chain rule again to find the dependence of the element loss at time t on the state at that same time,

$$\underbrace{\frac{\partial \mathcal{L}_{\text{elt}}(g_t, y_t)}{\partial s_t}}_{(m \times 1)} = \underbrace{\frac{\partial z_t^2}{\partial s_t}}_{(m \times \nu)} \underbrace{\frac{\partial \mathcal{L}_{\text{elt}}(g_t, y_t)}{\partial z_t^2}}_{(\nu \times 1)} , \tag{C.39}$$

and the dependence of the state at time t on the state at the previous time,

$$\underbrace{\frac{\partial s_t}{\partial s_{t-1}}}_{(m \times m)} = \underbrace{\frac{\partial z_t^1}{\partial s_{t-1}}}_{(m \times m)} \underbrace{\frac{\partial s_t}{\partial z_t^1}}_{(m \times m)} = W^{ssT} \frac{\partial s_t}{\partial z_t^1} \tag{C.40}$$

Note that $\partial s_t / \partial z_t^1$ is formally an $m \times m$ diagonal matrix, with the values along the diagonal being $f_s'(z_{t,i}^1)$, $1 \leqslant i \leqslant m$. But since this is a diagonal matrix, one could represent it as an $m \times 1$ vector $f_s'(z_t^1)$. In that case the product of the matrix $W^{ss\,T}$ by the vector $f_s'(z_t^1)$, denoted $W^{ss\,T} * f_s'(z_t^1)$, should be interpreted as follows: take the first column of the matrix $W^{ss\,T}$ and multiply each of its elements by the first element of the vector $\partial s_t / \partial z_t^1$, then take the second column of the matrix $W^{ss\,T}$ and multiply each of its elements by the second element of the vector $\partial s_t / \partial z_t^1$, and so on and so forth ...

Putting this all together, we end up with

$$\delta^{s_{t-1}} = \underbrace{W^{ss\,T} \frac{\partial s_t}{\partial z_t^1}}_{\frac{\partial s_t}{\partial s_{t-1}}} \underbrace{\left(W^{o\,T} \frac{\partial \mathcal{L}_t}{\partial z_t^2} + \delta^{s_t} \right)}_{\frac{\partial F_{t-1}}{\partial s_t}} \tag{C.41}$$

We're almost there! Now, we can describe the actual weight updates. Using Eq. C.32 and recalling the definition of $\delta^{s_t} = \partial F_t / \partial s_t$, as we iterate backwards, we can accumulate the terms in Eq. C.32 to get the gradient for the whole loss.

$$\frac{d\mathcal{L}_{seq}}{dW^{ss}} = \sum_{t=1}^{n} \frac{d\mathcal{L}_{elt}(g_t, y_t)}{dW^{ss}} = \sum_{t=1}^{n} \frac{\partial z_t^1}{\partial W^{ss}} \frac{\partial s_t}{\partial z_t^1} \frac{\partial F_{t-1}}{\partial s_t} \tag{C.42}$$

$$\frac{d\mathcal{L}_{seq}}{dW^{sx}} = \sum_{t=1}^{n} \frac{d\mathcal{L}_{elt}(g_t, y_t)}{dW^{sx}} = \sum_{t=1}^{n} \frac{\partial z_t^1}{\partial W^{sx}} \frac{\partial s_t}{\partial z_t^1} \frac{\partial F_{t-1}}{\partial s_t} \tag{C.43}$$

We can handle W^o separately; it's easier because it does not affect future losses in the way that the other weight matrices do:

$$\frac{d\mathcal{L}_{seq}}{dW^o} = \sum_{t=1}^{n} \frac{d\mathcal{L}_t}{dW^o} = \sum_{t=1}^{n} \frac{\partial \mathcal{L}_t}{\partial z_t^2} \frac{\partial z_t^2}{\partial W^o} \tag{C.44}$$

Assuming we have $\frac{\partial \mathcal{L}_t}{\partial z_t^2} = (g_t - y_t)$, (which ends up being true for squared loss, softmax-NLL, etc.), then

$$\underbrace{\frac{d\mathcal{L}_{seq}}{dW^o}}_{v \times m} = \sum_{t=1}^{n} \underbrace{(g_t - y_t)}_{v \times 1} \underbrace{s_t^T}_{1 \times m} . \tag{C.45}$$

Whew!

Study Question: Derive the updates for the offsets W_0^{ss} and W_0^o.

C.6 Vanishing gradients and gating mechanisms

Let's take a careful look at the backward propagation of the gradient along the sequence:

$$\delta^{s_{t-1}} = \frac{\partial s_t}{\partial s_{t-1}} \left[\frac{\partial \mathcal{L}_{elt}(g_t, y_t)}{\partial s_t} + \delta^{s_t} \right] . \tag{C.46}$$

Consider a case where only the output at the end of the sequence is incorrect, but it depends critically, via the weights, on the input at time 1. In this case, we will multiply the loss at step n by

$$\frac{\partial s_2}{\partial s_1}\frac{\partial s_3}{\partial s_2}\cdots\frac{\partial s_n}{\partial s_{n-1}} \quad . \tag{C.47}$$

In general, this quantity will either grow or shrink exponentially with the length of the sequence, and make it very difficult to train.

Study Question: The last time we talked about exploding and vanishing gradients, it was to justify per-weight adaptive step sizes. Why is that not a solution to the problem this time?

An important insight that really made recurrent networks work well on long sequences is the idea of *gating*.

C.6.1 Simple gated recurrent networks

A computer only ever updates some parts of its memory on each computation cycle. We can take this idea and use it to make our networks more able to retain state values over time and to make the gradients better-behaved. We will add a new component to our network, called a *gating network*. Let g_t be a $m \times 1$ vector of values and let W^{gx} and W^{gs} be $m \times l$ and $m \times m$ weight matrices, respectively. We will compute g_t as

$$g_t = \text{sigmoid}(W^{gx}x_t + W^{gs}s_{t-1}) \tag{C.48}$$

It can have an offset, too, but we are omitting it for simplicity.

and then change the computation of s_t to be

$$s_t = (1 - g_t) * s_{t-1} + g_t * f_s(W^{sx}x_t + W^{ss}s_{t-1} + W_0^{ss}) \ , \tag{C.49}$$

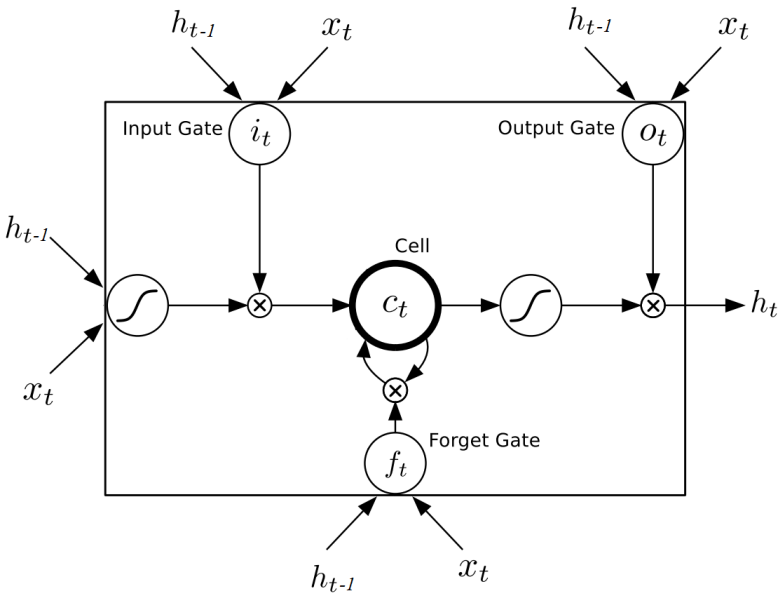
where $*$ is component-wise multiplication. We can see, here, that the output of the gating network is deciding, for each dimension of the state, how much it should be updated now. This mechanism makes it much easier for the network to learn to, for example, "store" some information in some dimension of the state, and then not change it during future state updates, or change it only under certain conditions on the input or other aspects of the state.

Study Question: Why is it important that the activation function for g be a sigmoid?

C.6.2 Long short-term memory

The idea of gating networks can be applied to make a state machine that is even more like a computer memory, resulting in a type of network called an LSTM for "long short-term memory." We won't go into the details here, but the basic idea is that there is a memory cell (really, our state vector) and three (!) gating networks. The *input* gate selects (using a "soft" selection as in the gated network above) which dimensions of the state will be updated with new values; the *forget* gate decides which dimensions of the state will have its old values moved toward 0, and the *output* gate decides which dimensions of the state will be used to compute the output value. These networks have been used in applications like language translation with really amazing results. A diagram of the architecture is shown below:

Yet another awesome name for a neural network!


h_{t-1}
x_t
h_{t-1}
x_t
Input Gate
i_t
Output Gate
o_t
h_{t-1}
Cell
c_t
h_t
x_t
Forget Gate
f_t
h_{t-1}
x_t

APPENDIX D

Supervised learning in a nutshell

In which we try to describe the outlines of the "lifecycle" of supervised learning, including hyperparameter tuning and evaluation of the final product.

D.1 General case

We start with a very generic setting.

D.1.1 Minimal problem specification

Given:

- Space of inputs $\mathcal{X}$
- Space of outputs $\mathcal{Y}$
- Space of possible *hypotheses* $\mathcal{H}$ such that each $h \in \mathcal{H}$ is a function $h : \mathcal{X} \to \mathcal{Y}$
- Loss function $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$

a *supervised learning algorithm* $\mathcal{A}$ takes as input a *data set* of the form

$$\mathcal{D} = \left\{ \left(x^{(1)}, y^{(1)} \right), \ldots, \left(x^{(n)}, y^{(n)} \right) \right\} \ ,$$

where $x^{(i)} \in \mathcal{X}$ and $y^{(i)} \in \mathcal{Y}$ and returns an $h \in \mathcal{H}$.

D.1.2 Evaluating a hypothesis

Given a problem specification and a set of data $\mathcal{D}$, we evaluate hypothesis h according to average loss, or *error*,

$$\mathcal{E}(h, \mathcal{L}, \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{\mathcal{D}} \mathcal{L}(h(x^{(i)}), y^{(i)})$$

If the data used for evaluation *were not used during learning of the hypothesis* then this is a reasonable estimate of how well the hypothesis will make additional predictions on new data from the same source.

D.1.3 Evaluating a supervised learning algorithm

A *validation strategy* $\mathcal{V}$ takes an algorithm $\mathcal{A}$, a loss function $\mathcal{L}$, and a data source $\mathcal{D}$ and produces a real number which measures how well $\mathcal{A}$ performs on data from that distribution.

D.1.3.1 Using a validation set

In the simplest case, we can divide $\mathcal{D}$ into two sets, $\mathcal{D}^{\text{train}}$ and $\mathcal{D}^{\text{val}}$, train on the first, and then evaluate the resulting hypothesis on the second. In that case,

$$\mathcal{V}(\mathcal{A}, \mathcal{L}, \mathcal{D}) = \mathcal{E}(\mathcal{A}(\mathcal{D}^{\text{train}}), \mathcal{L}, \mathcal{D}^{\text{val}}) \ .$$

D.1.3.2 Using multiple training/evaluation runs

We can't reliably evaluate an algorithm based on a single application of it to a single training and test set, because there are many aspects of the training and testing data, as well as, sometimes, randomness in the algorithm itself, that cause *variance* in the performance of the algorithm. To get a good idea of how well an algorithm performs, we need to, multiple times, train it and evaluate the resulting hypothesis, and report the average over K executions of the algorithm of the error of the hypothesis it produced each time.

We divide the data into $2K$ random non-overlapping subsets: $\mathcal{D}_1^{\text{train}}, \mathcal{D}_1^{\text{val}}, \ldots, \mathcal{D}_K^{\text{train}}, \mathcal{D}_K^{\text{val}}$. Then,

$$\mathcal{V}(\mathcal{A}, \mathcal{L}, \mathcal{D}) = \frac{1}{K} \sum_{k=1}^{K} \mathcal{E}(\mathcal{A}(\mathcal{D}_k^{\text{train}}), \mathcal{L}, \mathcal{D}_k^{\text{val}}) \ .$$

D.1.3.3 Cross validation

In *cross validation*, we do a similar computation, but allow data to be re-used in the K different iterations of training and testing the algorithm (but never share training and testing data for a single iteration!). See Section 2.7.2.2 for details.

D.1.4 Comparing supervised learning algorithms

Now, if we have two different algorithms $\mathcal{A}_1$ and $\mathcal{A}_2$, we might be interested in knowing which one will produce hypotheses that generalize the best, using data from a particular source. We could compute $\mathcal{V}(\mathcal{A}_1, \mathcal{L}, \mathcal{D})$ and $\mathcal{V}(\mathcal{A}_\in, \mathcal{L}, \mathcal{D})$, and prefer the algorithm with lower validation error. More generally, given algorithms $\mathcal{A}_1, \ldots, \mathcal{A}_M$, we would prefer

$$\mathcal{A}^* = \arg\min_m \mathcal{V}(\mathcal{A}_M, \mathcal{L}, \mathcal{D}) \ .$$

D.1.5 Fielding a hypothesis

Now what? We have to deliver a hypothesis to our customer. We now know how to find the algorithm, $\mathcal{A}^*$, that works best for our type of data. We can apply it to all of our data to get the best hypothesis we know how to create, which would be

$$h^* = \mathcal{A}^*(\mathcal{D}) \ ,$$

and deliver this resulting hypothesis as our best product.

D.1.6 Learning algorithms as optimizers

A majority of learning algorithms have the form of optimizing some objective involving the training data and a loss function.

So for example, (assuming a perfect optimizer which doesn't, of course, exist) we might say our algorithm is to solve an optimization problem:

$$\mathcal{A}(\mathcal{D}) = \arg\min_{h \in H} \mathcal{J}(h; \mathcal{D}) \quad .$$

Interestingly, this loss function is *not always the same* as the loss function that is used for evaluation! We will see this in logistic regression.

Our objective often has the form

$$\mathcal{J}(h; \mathcal{D}) = \mathcal{E}(h, \mathcal{L}, \mathcal{D}) + \mathcal{R}(h) \quad ,$$

where $\mathcal{L}$ is a loss to be minimized during training and $\mathcal{R}$ is a regularization term.

D.1.7 Hyperparameters

Often, rather than comparing an arbitrary collection of learning algorithms, we think of our learning algorithm as having some parameters that affect the way it maps data to a hypothesis. These are not parameters of the hypothesis itself, but rather parameters of the *algorithm*. We call these *hyperparameters*. A classic example would be to use a hyperparameter λ to govern the weight of a regularization term on an objective to be optimized:

$$\mathcal{J}(h; \mathcal{D}) = \mathcal{E}(h, \mathcal{L}, \mathcal{D}) + \lambda \mathcal{R}(h) \quad .$$

Then we could think of our algorithm as $\mathcal{A}(\mathcal{D}; \lambda)$. Picking a good value of λ is the same as comparing different supervised learning algorithms, which is accomplished by validating them and picking the best one!

D.2 Concrete case: linear regression

In linear regression the problem formulation is this:

- $\mathcal{X} = \mathbb{R}^d$
- $\mathcal{Y} = \mathbb{R}$
- $\mathcal{H} = \{\theta^T x + \theta_0\}$ for values of parameters $\theta \in \mathbb{R}^d$ and $\theta_0 \in \mathbb{R}$.
- $\mathcal{L}(g, y) = (g - y)^2$

Our learning algorithm has hyperparameter λ and can be written as:

$$\mathcal{A}(\mathcal{D}; \lambda) = \Theta^*(\lambda, \mathcal{D}) = \arg\min_{\theta, \theta_0} \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} (\theta^T x + \theta_0 - y)^2 + \lambda \|\theta\|^2 \quad .$$

For a particular training data set and parameter λ, it finds the best hypothesis on this data, specified with parameters $\Theta = (\theta, \theta_0)$, written $\Theta^*(\lambda, \mathcal{D})$.

Picking the best value of the hyperparameter is choosing among learning algorithms. We could, most simply, optimize using a single training / validation split, so $\mathcal{D} = \mathcal{D}^{\text{train}} \cup \mathcal{D}^{\text{val}}$, and

$$\begin{aligned}
\lambda^* &= \arg\min_{\lambda} \mathcal{V}(\mathcal{A}_\lambda, \mathcal{L}, \mathcal{D}^{\text{val}}) \\
&= \arg\min_{\lambda} \mathcal{E}(\Theta^*(\lambda, \mathcal{D}^{\text{train}}), \text{mse}, \mathcal{D}^{\text{val}}) \\
&= \arg\min_{\lambda} \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{(x,y) \in \mathcal{D}_{\text{val}}} (\theta^*(\lambda, \mathcal{D}^{\text{train}})^T x + \theta_0^*(\lambda, \mathcal{D}^{\text{train}}) - y)^2
\end{aligned}$$

It would be much better to select the best λ using multiple runs or cross-validation; that would just be a different choices of the validation procedure $\mathcal{V}$ in the top line.

Note that we don't use regularization here because we just want to measure how good the output of the algorithm is at predicting values of *new* points, and so that's what we measure. We use the regularizer during training when we don't want to focus only on optimizing predictions on the training data.

Finally! To make a predictor to ship out into the world, we would use all the data we have, $\mathcal{D}$, to train, using the best hyperparameters we know, and return

$$\begin{aligned}\Theta^* &= \mathcal{A}(\mathcal{D}; \lambda^*) \\ &= \Theta^*(\lambda^*, \mathcal{D}) \\ &= \arg\min_{\theta, \theta_0} \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} (\theta^T x + \theta_0 - y)^2 + \lambda^* \|\theta\|^2\end{aligned}$$

Finally, a customer might evaluate this hypothesis on their data, which we have never seen during training or validation, as

$$\begin{aligned}\mathcal{E}^{\text{test}} &= \mathcal{E}(\Theta^*, \text{mse}, \mathcal{D}^{\text{test}}) \\ &= \frac{1}{|\mathcal{D}^{\text{test}}|} \sum_{(x,y)\in\mathcal{D}^{\text{test}}} (\theta^{*T} x + \theta_0^* - y)^2\end{aligned}$$

Here are the same ideas, written out in informal pseudocode.

```
# returns theta_best(D, lambda)
define train(D, lambda):
    return minimize(mse(theta, D) + lambda * norm(theta)**2, theta)

# returns lambda_best using very simple validation
define simple_tune(D_train, D_val, possible_lambda_vals):
    scores = [mse(train(D_train, lambda), D_val)
                  for lambda in possible_lambda_vals]
    return possible_lambda_vals[least_index[scores]]

# returns theta_best overall
define theta_best(D_train, D_val, possible_lambda_vals):
  return train(D_train + D_val,
               simple_tune(D_train, D_val, possible_lambda_vals))

# customer evaluation of the theta delivered to them
define customer_val(theta):
    return mse(theta, D_test)
```

D.3 Concrete case: logistic regression

In binary logistic regression the problem formulation is as follows. We are writing the class labels as 1 and 0.

- $\mathcal{X} = \mathbb{R}^d$

- $\mathcal{Y} = \{+1, 0\}$
- $\mathcal{H} = \{\sigma(\theta^T x + \theta_0)\}$ for values of parameters $\theta \in \mathbb{R}^d$ and $\theta_0 \in \mathbb{R}$.
- $\mathcal{L}(g, y) = \mathcal{L}_{01}(g, h)$
- Proxy loss $\mathcal{L}_{\text{nll}}(g, y) = -(y \log(g) + (1 - y) \log(1 - g))$

Our learning algorithm has hyperparameter λ and can be written as:

$$\mathcal{A}(\mathcal{D}; \lambda) = \Theta^*(\lambda, \mathcal{D}) = \arg\min_{\theta, \theta_0} \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} \mathcal{L}_{\text{nll}}(\sigma(\theta^T x + \theta_0), y) + \lambda \|\theta\|^2 \quad .$$

For a particular training data set and parameter λ, it finds the best hypothesis on this data, specified with parameters $\Theta = (\theta, \theta_0)$, written $\Theta^*(\lambda, \mathcal{D})$ according to the proxy loss $\mathcal{L}_{\text{nll}}$.

Picking the best value of the hyperparameter is choosing among learning algorithms based on their actual predictions. We could, most simply, optimize using a single training / validation split, so $\mathcal{D} = \mathcal{D}^{\text{train}} \cup \mathcal{D}^{\text{val}}$, and *we use the real 01 loss*:

$$\begin{aligned}
\lambda^* &= \arg\min_{\lambda} \mathcal{V}(\mathcal{A}_\lambda, \mathcal{L}_{01}, \mathcal{D}^{\text{val}}) \\
&= \arg\min_{\lambda} \mathcal{E}(\Theta^*(\lambda, \mathcal{D}^{\text{train}}), \mathcal{L}_{01}, \mathcal{D}^{\text{val}}) \\
&= \arg\min_{\lambda} \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{(x,y) \in \mathcal{D}_{\text{val}}} \mathcal{L}_{01}(\sigma(\theta^*(\lambda, \mathcal{D}^{\text{train}})^T x + \theta_0^*(\lambda, \mathcal{D}^{\text{train}})), y)
\end{aligned}$$

It would be much better to select the best λ using multiple runs or cross-validation; that would just be a different choices of the validation procedure $\mathcal{V}$ in the top line.

Finally! To make a predictor to ship out into the world, we would use all the data we have, $\mathcal{D}$, to train, using the best hyperparameters we know, and return

$$\Theta^* = \mathcal{A}(\mathcal{D}; \lambda^*)$$

Study Question: What loss function is being optimized inside this algorithm?

Finally, a customer might evaluate this hypothesis on their data, which we have never seen during training or validation, as

$$\mathcal{E}^{\text{test}} = \mathcal{E}(\Theta^*, \mathcal{L}_{01}, \mathcal{D}^{\text{test}})$$

The customer just wants to buy the right stocks! So we use the real $\mathcal{L}_{01}$ here for validation.

Index